

Compact Data Structures for Network Telemetry

SHIR LANDAU-FEIBISH, Computer Science, The Open University of Israel, Raanana, Israel

ZAOXING LIU, Computer Science, University of Maryland, College Park, United States

JENNIFER REXFORD, Princeton University, Princeton, United States

Collecting and analyzing of network traffic data (*network telemetry*) plays a critical role in managing modern networks. Network administrators analyze their traffic to troubleshoot performance and reliability problems and to detect and block cyberattacks. However, conventional traffic-measurement techniques offer limited visibility into network conditions and rely on offline analysis. Fortunately, network devices—such as switches and network interface cards—are increasingly programmable at the packet level, enabling flexible analysis of the traffic in place, as the packets fly by. However, to operate at high speed, these devices have limited memory and computational resources, leading to trade-offs between accuracy and overhead. In response, an exciting research area emerged, bringing ideas from compact data structures and streaming algorithms to bear on important networking telemetry applications and the unique characteristics of high-speed network devices. In this article, we review the research on compact data structures for network telemetry and discuss promising directions for future research.

CCS Concepts: • **Networks** → **Programmable networks**; **Data path algorithms**; **Network monitoring**;

ACM Reference Format:

Shir Landau-Feibish, Zaoxing Liu, and Jennifer Rexford. 2025. Compact Data Structures for Network Telemetry. *ACM Comput. Surv.* 57, 8, Article 191 (March 2025), 31 pages. <https://doi.org/10.1145/3716819>

1 Introduction

Network administrators rely on traffic measurements to manage performance problems, flaky equipment, and cyberattacks in their networks. For example, traffic measurements can reveal unusual levels of packet loss and delay, indicative of network congestion. Similarly, traffic measurements can show a host receiving traffic from many different senders, suggestive of a **distributed denial-of-service (DDoS)** attack. *Network telemetry*—collecting and analyzing traffic measurements—enables network administrators to diagnose these problems, such as identifying the traffic responsible for congestion or pinpointing the senders participating in a denial-of-service attack. Measurement data also help network administrators model the effects of proposed configuration changes to alleviate these problems, such as redirecting some traffic onto a different path or dropping packets from suspected attackers.

This work is supported in part by the Israel Science Foundation (Grant No. 980/21) and the U.S. National Science Foundation (Grant No. CNS-1704077). Liu is supported in part by the U.S. National Science Foundation CNS-2431093.

Authors' Contact Information: Shir Landau-Feibish, Computer Science, The Open University of Israel, Raanana, Israel; e-mail: lfshir@gmail.com; Zaoxing Liu, Computer Science, University of Maryland, College Park, Maryland, United States; e-mail: zaoxing@umd.edu; Jennifer Rexford, Princeton University, Princeton, New Jersey, United States; e-mail: jrex@cs.princeton.edu.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

© 2025 Copyright held by the owner/author(s).

ACM 0360-0300/2025/03-ART191

<https://doi.org/10.1145/3716819>

1.1 Traditional Traffic Measurement

Unfortunately, traditional high-speed network devices offer only limited visibility into the traffic, due to the overheads of collecting and exporting the measurement data for subsequent analysis. For example, each link may report statistics like utilization and packet loss, using the **Simple Network Management Protocol (SNMP)**, but only on the timescale of minutes. In addition, devices may report more detailed packet-level information (using technologies like Netflow [40], sFlow [122], and IPFIX [41, 99]), but only for a small fraction, such as 1 in 5,000, of the packets [114].

These measurements provide a high-level summary of network conditions, but they do not offer the timely, fine-grained information needed to drive real-time management decisions. For example, SNMP data can show that a link suffers from persistent congestion, but not that a microburst disrupted performance for a few tens of milliseconds. Similarly, Netflow data can identify the applications contributing the most traffic, but not performance statistics (like round-trip times or the prevalence of packet reordering) that look across multiple packets in the same flow. In addition, sending measurement data to a collector for analysis introduces delay in reacting to changes in the network. Fast reactions are important for alleviating congestion for real-time applications (such as video conferencing or self-driving cars) or blocking cyberattacks that are overwhelming victims.

Perhaps most importantly, traditional measurement techniques cannot be *customized* for the telemetry task at hand. SNMP and Netflow are useful for a variety of purposes, but they are not the best solution for any one telemetry task. Sometimes, the total traffic volume on a 1-min timescale (as in SNMP) is the right statistic, but often it is not. Similarly, sometimes packet samples of particular header fields (as in NetFlow) are the right data, but often they are not. Network administrators need effective ways to customize the measurements they collect and the analysis they do. They need effective ways to extract specific information from each packet, and combine that information across successive packets to have fine-grained visibility into network conditions at scale.

1.2 Programmable Network Devices

The emergence of programmable network devices, including switches and **network interface cards (NICs)**, is poised to change all that. Network devices have long run software in the *control plane* to track changes to the network topology and compute routes. Now, the *data plane* of network devices is increasingly programmable at the level of individual packets, with flexible parsing and computation based on packet header fields, as well as memory for accumulating information across successive packets. A prominent early example is the Reconfigurable Match Table architecture [26] that was the basis for the Intel Tofino chipset [5], which has been a popular platform in the networking research community. Other examples of programmable switches include Broadcom Trident [4], Juniper Trio [127], Aruba CX 10000 [3], and Xsight X2 [11]. These switches are programmed using domain-specific languages like P4 [25, 117] and NPL [7]. In addition, programmable (*smart*) NICs are available from various vendors, including Netronome [1], Pensando [8], Mellanox [6], and Xilinx [2]. Whereas high-speed switches often use programmable ASICs (application-specific integrated circuits), SmartNICs often use technologies such as Field Programmable Gate Arrays and multi-core engines that offer greater flexibility but necessarily operate at lower speed.

Flexible packet processing on network devices enables network administrators to customize the measurement data to the task at hand by collecting, analyzing, and even acting on the measurement results directly as the packets fly by. For example, a network switch could identify the traffic responsible for a backlogged queue, and mark or drop the offending packets to alleviate the congestion. As another example, the switch could identify the IP addresses of servers sending response traffic that does not correspond to any recent client request, and drop the unsolicited

traffic. As yet another example, the switch could identify senders contacting a large number of distinct receivers, or receivers contacted by a large number of distinct senders, to detect and mitigate denial-of-service attacks. These and other telemetry tasks capitalize on programmability to group related packets with common header fields into a *flow* and then compute and store per-flow statistics ranging from simple counts to more sophisticated performance and security metrics.

Unfortunately, high-speed network devices have significant resource limitations. These devices are domain-specific processors designed to process packets at high speed to keep up with link capacity. As such, these devices can only perform simple operations and maintain limited state. Plus, since memory bandwidth is not keeping pace with link speed, the number of *accesses* to memory for each packet is limited. In practice, many of these devices consist of a sequence of pipeline stages, each with match-action tables (for pattern matching on packet header fields), small register arrays (for storing data across successive packets), and simple arithmetic logic units (for performing addition, subtraction, and bit-wise operations). Moreover, these devices must devote many of these resources to perform routine packet forwarding, leaving fewer resources available for traffic measurement.

1.3 Compact Data Structures for Telemetry

Luckily, many telemetry questions do not require exact answers; often a reasonable estimate is fine. For example, identifying the most heavy flows may be more important than knowing the exact number of bytes or packets in these flows, let alone the sizes of the many smaller flows. Often network administrators care about a small fraction of the flows—the outliers—and a rough estimate of the associated statistics. Network administrators can exploit this tolerance by using *approximate* data structures that trade accuracy for lower measurement overhead. There is a long and rich history in the theoretical computer science community of research on compact data structures that can help design approximate solutions that “fit” in the data plane. However, past research on compact data structures does not always apply directly to high-speed programmable data planes. In particular, the constraints on the number of memory accesses, and the division of memory and processing across stages, do not arise in most earlier research on compact data structures.

Over the past few years, we have seen great progress in designing compact data structures for high-speed network devices. Many of these designs are variants of earlier compact data structures, tailored to the unique constraints of high-speed packet processing. Recent work shows how to support a wide range of important measurement tasks, both within a single network device and across a larger network. In this article, we present a survey of recent research on compact data structures for network telemetry, with an emphasis on how to grapple effectively with the unique constraints of modern network devices. Our goal is to reach both the networking and the theory communities to foster further interdisciplinary collaborations in this area. The article exposes theoretical computer scientists to a distinctive computational model for streaming algorithms, as well as a class of practical telemetry problems that need further study. For networking researchers, the article puts a large body of recent research in a common context and shows how to adapt algorithms to the constraints of high-speed network devices. This is the first survey to focus on the data structures used for telemetry in programmable network hardware. Existing surveys on telemetry in the data plane have focused on other aspects of telemetry such as in-band telemetry that tags packets with measurement statistics [85, 115]. Broader surveys exist that look at the hardware and applications of programmable networks [87, 134], or the P4 language [65, 70], that provide a wide-ranging discussion of telemetry in the data plane. Furthermore, surveys on sketches in general have also been presented [29, 44]. Our goal is to look at the overlap of these areas, to provide a much more extensive exploration of the data structures used for network telemetry to highlight both the challenges and considerations in creating and using these structures within programmable devices.

The remainder of the article is structured as follows. In Section 2, we discuss the goals of network telemetry, and introduce a broad class of measurement tasks that perform queries on packet tuples. Then, Section 3 makes the case for supporting these tasks directly in the data plane, and introduces a computational model for high-speed network devices. Section 4 starts our discussion of compact data structures for these devices, for simple queries that estimate set membership and per-flow traffic counts. Then, Section 5 considers more sophisticated queries for anomaly detection and performance monitoring. Section 6 delves further into the practical challenges of allocating data-plane resources, to manage the trade-off between accuracy and overhead. The next two sections discuss early research in two promising directions: distributed network telemetry (Section 7) and robustness to adversaries who try to manipulate the measurement process (Section 8). The article concludes in Section 9.

2 Network Telemetry Queries

Each packet in the network can be thought of as a *tuple* of header fields (e.g., source IP address, destination port number, TCP sequence number, etc.) and relevant attributes (e.g., packet size, timestamp, location, or queue length traversed). A telemetry query runs over a stream of tuples by applying database-like operators inspired by platforms such as SQL or Map-Reduce [61, 84, 94, 143].

These queries often group related packets into a single *flow*, such as a TCP connection or traffic between the source and destination IP prefixes. Precisely, a flow is a set of packets that share the same *flow identifier*, which is defined as packets sharing some tuple fields in common (e.g., the same 5-tuple, which consists of the transport protocol, the source and destination IP addresses, and the source and destination port numbers). A flow telemetry query calculates one or more *metrics* based on the attribute associated with each flow and performs an aggregation on the packets of the flow, e.g., summation over packet count, bytes, or distinct number of flows.

In addition, network administrators often perform telemetry queries on performance-related attributes, such as the packet timestamp. Estimating such statistics (e.g., round-trip latency) requires combining information across pairs of packets with stateful operations (e.g., the average round-trip time between a request packet and its acknowledgment). The traffic measured in telemetry tasks is often defined by various time windows, or *epochs*, where an epoch represents a time period (e.g., several seconds to minutes). Table 2 summarizes common telemetry queries and their applications.

2.1 Volumetric and Aggregated Flow Statistics

Volumetric Flow Queries. A common set of telemetry queries focus on the *size* of a flow, such as the number of packets or the total byte count. With unlimited memory and compute resources, we could compute and store the sizes of all flows. However, network traffic is too large to be recorded at the per-flow level. Traditional telemetry approaches such as SNMP and NetFlow are usually too coarse-grained and report the measurements only on large epochs with sampled packets. Thus, the most popular queries in this class are α -heavy hitters and Top-K flows.

- α -Heavy hitters: (also known as *elephant* flows if measuring packet byte counts) are the large flows that consume more than a fraction α of the total traffic capacity. Network administrators can specify a fixed threshold α beforehand or set dynamic thresholds during the measurement.
- Top-K flows: are a variant of the heavy-hitter problem that reports the K largest flows in the stream.

Volumetric flow queries are useful for a number of downstream network-management applications as heavy hitters are essential for network performance and security optimizations. For

Table 1. Flow Statistics and Queries

Metric	Definition	Application
α -Heavy hitters	All flows f_i s.t. $ f_i \geq \alpha \cdot n$	Traffic engineering [53]
Top-K flows	$\{f_1, f_2, \dots, f_k\}$	Load balancer [80]
Distinct count (cardinality)	The size of the set $\{f_1, f_2, \dots, f_n\} : f_i \in S$	Attack detection [130]
Entropy	$-\sum_{i=1}^j \frac{f_i}{n} \log(\frac{f_i}{n})$	Anomaly detection [81]

Given a stream of packets S , with n packets and j distinct flows. Let $|f_i|$ denote the frequency of the i th unique element in the stream s.t. $f_1 \geq f_2 \geq \dots \geq f_j$.

instance, traffic engineering [53] needs to identify the largest flows and prioritize them to meet the service-level agreements. Similarly, in events of volumetric DDoS attacks [82], measuring heavy-hitter flows can guide further investigation.

Aggregated flow statistics. With the assumption of not recording the information for all flows, network operators also need to compute various aggregated statistical metrics that summarize all the flows. These metrics, such as entropy, quantiles, and distinct count, are concise representations of the flow size and metadata distributions, and are useful to monitor overall network conditions.

- Distinct count: the number of *unique* elements seen in a stream. This can be a count of the number of different flows seen or the number of different attributes seen for a given flow. For example, for a given destination IP, we can measure the number of unique source IPs from which it is receiving traffic. A relatively high number of distinct source IPs could indicate that the destination is experiencing a DDoS attack. Moreover, the change in the number of distinct flows (i.e., the number of unique flows in a stream of packets) is a strong indicator for flash events (e.g., short-term traffic bursts) or ongoing DDoS attacks [35, 82, 128].
- Quantiles: the general term for metrics such as median, 95th or 99th percentile. More broadly, quantiles can be inferred from the cumulative distribution function. Quantiles are useful for identifying both short-term anomalies and long-term changes in the traffic [68]
- Entropy: a measure of the diversity of the traffic [74] that can be used as an indicator for the distribution of the traffic. Anomaly detection may look into the distributions over different flow identifiers (e.g., source IP, destination IP) via continuously computing their entropy values [96] and identifying their changes [81].

Some of the metrics presented above are formally defined in Table 1.

2.2 Queries Over Packet Metadata

When packets traverse a network, multiple performance-related metrics can be recorded and attached to the packets as metadata, e.g., timestamps, switch locations, and queue lengths. These metadata can be used to compute various useful metrics about network performance. For instance, by calculating the timestamp difference between outgoing and incoming traffic of a flow, we can estimate the round-trip time of the flow to a remote destination. We can also identify if packets in a flow are seen out-of-order, which would normally cause retransmission and delays, possibly creating jitter and affecting the quality of experience of the network users. Moreover, by obtaining the queue occupancy information from each switch a packet traverses, we can understand the congestion status precisely and optimize the configurations for better performance accordingly.

2.3 Queries Over Measurement Windows

Since network measurement data becomes less relevant with time, telemetry systems typically produce statistics about the most recently seen traffic. For example, a network operator may want

Table 2. Example Queries Over Packet Metadata and Applications

Attribute	Metric	Application
Seq. No.	Out-of-order packets	Quality of experience (QoE) [83]
Timestamp	Round-trip times	QoE/Congestion [101]
Timestamp	TTL changes	Network memory allocation [111]
Key	Set membership	Rate Limit [82]
Queue Length	Large queues	Congestion Control [77]

to know the top-ten flows over a 30-s period. With a *tumbling* window, the time intervals do not overlap, and each packet is processed once and belongs to one window. For example, the first interval would correspond to times 0 to 30 s, while the next corresponds to times 30 to 60 s. In contrast, a *sliding* window slides over the stream of packets, always maintaining statistics for the most recent packets. Sliding windows are more expensive to maintain, since the older packets expire gradually. In practice, a sliding window may be approximated using multiple smaller tumbling windows (e.g., times 0–10, 10–20, and 20–30 for the 0–30 s interval, followed by times 10–20, 20–30, and 30–40 for the next interval covering times 10–40).

3 Case for Data-plane Telemetry

Although traditional measurement techniques have significant limitations, the emergence of programmable network devices enables unprecedented visibility into the underlying traffic. Programmable data planes enable the *customization* of telemetry at the packet level, for efficient fine-grained analysis and real-time adaptation to changing conditions. However, to process packets at high speed, modern data planes impose significant limitations on memory and computational resources. Telemetry queries must “fit” within these resource constraints, as otherwise the data plane cannot serve traffic at line rate.

3.1 Programmable Packet Processing

Network devices, such as switches and NICs, are increasingly programmable at the packet level. These devices offer flexible:

- *parsing* of packets, to extract specific fields of interest,
- *matching* on these fields to group related packets into a single “flow,”
- *computation*, such as simple hash functions as well as other arithmetic and logic operations,
- *storage* of information across successive packets, and
- *communication* with a software controller.

Together, these capabilities make the data plane a simple kind of *stream processor* that performs operations over a sequence of packets. The programmable parser determines the “tuple” of fields extracted from each packet, the computation determines how to manipulate and update this data as the packet flies by, and the storage enables more sophisticated operations over multiple packets in the same flow. The data plane can generate reports to a software controller, and have the controller read the data-plane state and update the data-plane configuration. These data planes are programmable using domain-specific languages such as P4 [25, 117] and NPL [7].

Programmable data planes are a promising way to enable customized telemetry, which offers several important advantages over traditional measurement techniques. First, performing both the measurement and analysis in the data plane improves *efficiency*. The data plane can perform fine-grained analysis right as packets fly by, without exporting a large amount of data to the collector. Second, the data plane can incorporate local *metadata*, such as the current time or the current length of packet queues, into the analysis of the traffic. Third, the data plane can take timely

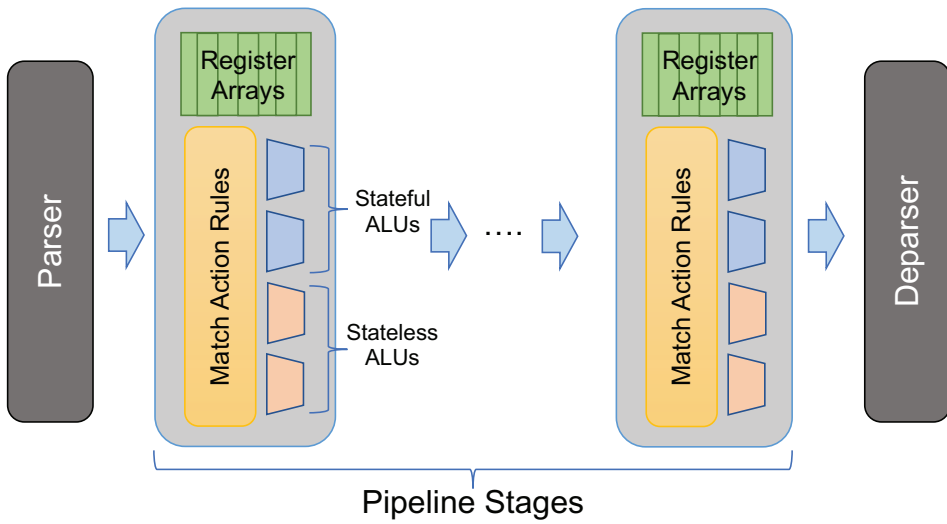


Fig. 1. Protocol-independent Switch Architecture (PISA) Data Plane.

action on individual packets based on the results of the analysis. For example, the data plane can drop, forward, or modify a packet in flight based on the results of the computation. Fourth, the data plane can protect *privacy* by computing the answers to measurement questions without ever exporting the raw data used in the computation.

3.2 Data-plane Resource Constraints

Unfortunately, programmable data planes have a number of limitations, due to the need to process packets at high speed. Typically, a high-speed data plane cannot parse arbitrarily deep into the packet, and computation is limited to simple arithmetic operations (e.g., addition and subtraction, but not multiplication and division) and logic operations (e.g., bit shifting). Since increases in memory bandwidth have not kept pace with link bandwidth, high-speed packet processing must work with limited memory resources. The memory is typically too small to store per-flow state. Plus, each packet can access the memory at most a small, constant number of times.

The exact constraints differ from one kind of network device to another, but all of these devices have these kinds of limitations, because high-speed links must be able to process a packet every few nanoseconds. A common computational model is the **Protocol-independent Switch Architecture (PISA)** where the data plane consists of a packet-processing pipeline with multiple stages of memory and processing resources. As shown in Figure 1, each stage has:

- small match-action tables that can perform exact or ternary (i.e., matching to 0, 1, or a wild-card matching either) matching of packet header fields based on rules installed by the control plane,
- simple arithmetic and logic units that perform actions, and
- small register arrays for storing information across successive packets,

That is, the register memory is partitioned across the stages, where memory in an earlier stage cannot be updated based on the results of computations at a later stage, unless the packet is *recirculated* to traverse the pipeline a second time. Each stage may have multiple parallel arithmetic/logic units and register arrays, allowing a single stage to perform multiple operations concurrently in the absence of dependencies. Note that ALUs may be stateful or stateless. Stateful ALUs can perform

actions on the register arrays in the memory of the stage. Stateless ALUs perform actions that do not access the registers in the stage, yet they may modify the packet metadata.

Figure 1 presents a single pipeline. The pipeline is coupled to several physical ports in the switch. Furthermore, programmable switches normally have multiple pipelines such that each of the ports is coupled to a dedicated pipeline. An internal traffic manager forwards the packets between the pipelines. Packets may also be recirculated back to the beginning of a pipeline. The data plane typically has limited bandwidth for recirculating packets, as well as limited bandwidth for communicating with the control-plane software.

Unfortunately, the resource constraints limit the accuracy of telemetry applications. Fortunately, most telemetry applications are robust to small errors, enabling the use of approximate data structures that can work reasonably well with a limited amount of memory and a limited number of memory accesses per packet. For example, a network administrator wanting to identify the heaviest flows may not mind if the estimates of per-flow traffic volumes have some errors.

4 Classic Data Structures

In this section, we give an overview of the data structures that are commonly used in network telemetry applications. Our goal is both to acquaint networking practitioners with these classic data structures and to introduce theorists to the challenges of realizing these structures in the data plane.

Many telemetry tasks rely on “counting” traffic volume (e.g., the number of bytes or packets) by flow, whether to estimate the count for each flow or to identify the heavy flows. Another common building block is “set membership,” where we need to represent a set of flows. To avoid maintaining per-flow state, the data plane must store an approximate summary of the “counts” or the “set.”

The approximation usually takes one of two forms: compressing all of the information (using a sketch) or discarding some information (using a cache). These techniques have different characteristics, which can guide which data structure to apply to a particular setting. Cache-based approaches typically store the key for each cached entry, making the key easy to retrieve after the fact; sketches do not. Sketches are compact summaries of a data set that can provide an approximate answer (e.g., an estimated count) on demand for any key; cache-based structures only produce estimates for the cached keys. The data structures also differ in their accuracy, as well as whether they consistently overestimate or underestimate the statistic of interest.

Adapting these data structures to the constraints of high-speed data planes can be challenging. In the rest of this section, we first present the data structures that are the easiest to realize in the data plane, followed by those that require more substantial modifications. We first discuss two common sketches, the count-min sketch (for counting) and the Bloom filter (for set membership), followed by Space Saving that caches the large flows (for identifying the heavy-hitters).

4.1 Sketch: Count-min and Bloom Filter

4.1.1 Count-min Sketch. To identify flows with counts that exceed a certain threshold, various sketches have been proposed including the count sketch [31], the count-min sketch [45], and the CU sketch [53]. The count-min sketch [45] is the de facto standard in programmable data planes, and is used extensively in telemetry applications [20, 36, 63, 79, 82, 83, 125]. As shown in Figure 2, the count-min sketch is a matrix with r rows and c columns, where each of the $r * c$ entries stores a count. A set of r independent hash functions is used to select an entry (a_i) in each row (i) for a given key. When updating the sketch, the associated counters of these indices are incremented. To estimate the count for a given key, the same hash functions are used to compute the same r indices. The estimated count is the *minimum* of the r values.

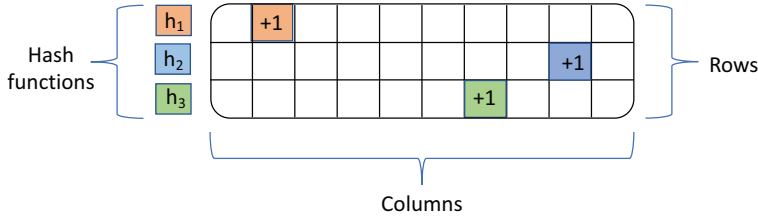


Fig. 2. Count-min Sketch. This is a sketch with $r = 3$ rows and $c = 10$ columns. Upon inserting a key it will be hashed by each of the hash functions and the relevant indices will be incremented by 1.

The counters in the count-min sketch provide an approximate count for each key, yet this approximation may incur errors. Errors in the count-min sketch are due to collisions. Namely, if more than one flow maps to a certain index, then the count is incremented when any one of these flows is seen, leading to over-estimation. Note that the error is one-sided; that is, the counters may only *over-estimate* the count but they never *under-estimate* the count. Therefore, while the count-min sketch can answer count-queries for medium-sized flows and not just heavy flows, smaller flows may have higher error rates, since they may be more significantly impacted by collisions with heavy flows. An interesting property of the count-min sketch is that it is linearly mergeable, meaning that several sketches can be combined into a single aggregated sketch. This property is especially useful for distributed telemetry, as discussed in Section 7.

Count-min sketches in the data plane. Generally speaking sketches fit very nicely within the constraints of the data plane. Each row of the sketch can be implemented as a register array within the memory of the switch. The number of columns is limited by the available memory. The number of hash functions, and thus the number of rows, which can be supported is limited by both the number of hash units available and the number of available memory accesses (since each row needs to be accessed exactly once when performing insert or get-count).

Count-min sketches are very space efficient structures. They do not require maintaining the keys of the flows, which could consume a lot of memory, and thus the size of the sketch remains constant, regardless of the size of the keys used. This is especially important in the data plane, since memory is statically allocated and dynamic allocation usually requires reconfiguring the switch. Nonetheless, the count-min sketch can only be queried if the key of the flow is known. If the key is not known, then it cannot be hashed, and thus information about the flow cannot be retrieved. Therefore, the count-min sketch is often used to find the count of a flow as one of its packets is processed.

Flow changes. The count-min sketch can also be used to identify flows that contribute the most to traffic change over two consecutive time windows. Consider two adjacent time windows t_A and t_B . The size of a flow i in t_A is $S_A[i]$ and $S_B[i]$ in t_B . The difference signal for x is defined as $D[x] = |S_A[i] - S_B[i]|$. A flow is a *heavy* change flow if the difference in its signal exceeds the ϕ percentage of the total change over all flows. The total difference is $D = \sum_{i \in [n]} D[i]$. A flow i is defined to be a heavy change iff $D[i] \geq \phi \cdot D$. To detect such flow changes, we can take advantage of the intrinsic *linearity* in count-min sketch and count sketch [45], which measure two arbitrary time windows. When querying the flow changes, we need to perform a *subtraction* on each of the counters between two sketch instances and obtain the heavy hitters among the change in traffic volume.

4.1.2 Bloom Filter. Another basic structure that is needed when processing traffic is a structure that maintains a set of flows, and supports set-membership queries. Due to the memory restrictions in the data plane, sets cannot be maintained in their entirety. Instead, we can use a sketch called

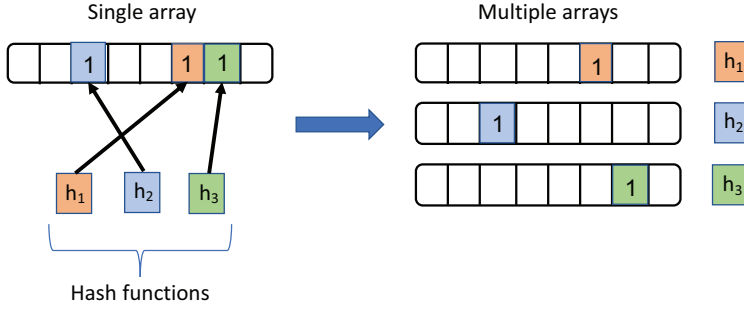


Fig. 3. Bloom filter in the data plane. A bloom filter composed of a single array using three hash functions will be separated to three separate arrays in the data plane, and each hash function will be used to access one dedicated array.

the Bloom filter [29]. A Bloom filter is a sub-linear sketch composed of a single array consisting of c bits, with h independent hash functions that are associated with the structure. When processing an item, each of the h hash functions is invoked on the key of the item to get h different indices in the array. To insert an element into the structure, each of the associated bits is set to 1. To perform a *find* operation on an item, the same indices are checked. If all of them are set to 1, then the find operation returns true. Otherwise, it returns false. A Bloom filter is always able to correctly identify items that have been inserted to the structure and therefore does not have any false negatives. However, due to collisions, an item may appear to be in the set, even though it was never inserted to the structure; hence, false positives are possible.

Bloom filters in the data plane. While Bloom filters may seem to be a simpler structure (or just as simple) than a count-min sketch, surprisingly, implementing them in the data plane requires more adaptations. The main issue that arises is that both insert and find operations on a Bloom filters require setting or checking several indices in the array, and thus require *multiple* accesses to the *same* array. The memory model of the switch makes this impossible in a single iteration of packet processing. To enable the use of multiple hash functions, the Bloom filter implementation in the data plane maintains a separate array for *each* hash function. As shown in Figure 3, for a Bloom filter with c bits using $h = 3$ hash functions, three separate arrays of size c bits need to be maintained. This results in a slightly different implementation than the standard Bloom filter. However, since it avoids collisions between the different hash functions, it potentially has fewer collisions. Thus providing an error rate that is no higher than the error rate provided by the regular implementation of Bloom filter. As with the count-min sketch, the parameters of the Bloom filter are constrained by the available resources.

4.2 Cache: Space Saving

One of the most basic structures is the key-value store. It is essentially a fixed-size hash table indexed by hashing a key, such that it effectively becomes a cache in a setting with limited memory, which can be used for both flow counts and set-membership. Upon insertion, an item is hashed to one index in the hash table. If the entry is empty, then the item is inserted. Otherwise, the item may evict an existing item or may be discarded. In either case, an item from the set would not appear or be counted in the structure, and thus may create false-negatives in set-membership or under-estimation in flow count approximations. Hash tables may be implemented as a single table or as a table divided across multiple stages to enable better handling of collisions, though multi-stage tables do not eliminate the problem.

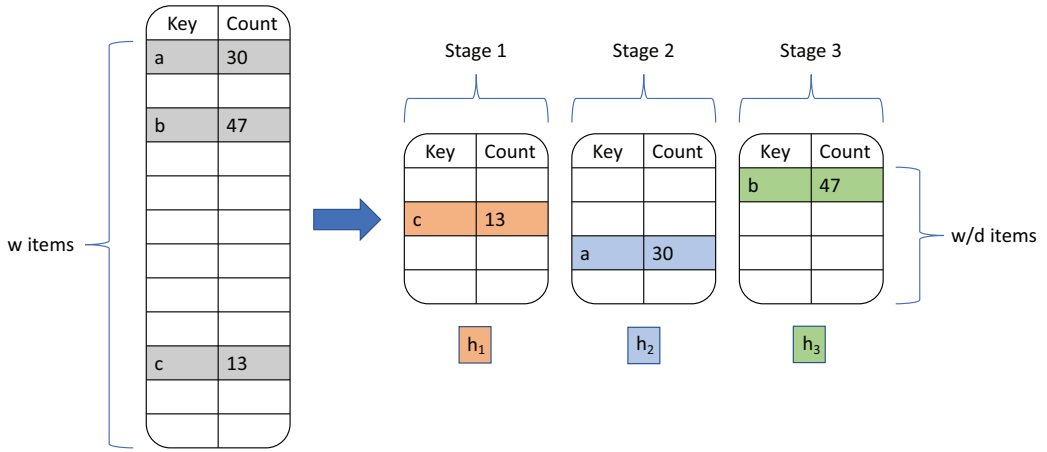


Fig. 4. Space Saving in the data plane. The Space Saving table of size w is divided into w/d tables for $d = 3$, with one part of the table in each stage. Items are hashed into each of the tables and thus may appear in any of d tables.

Space Saving. There are several variants for the flow count problem, such as finding the heavy-hitter flows or the top- k flows. Several cache-based (also known as counter-based) algorithms exist in the theory literature for solving the heavy-hitter problem. Perhaps, the most widely used algorithm is the Space Saving algorithm of Metwally et al. [86]. Space Saving maintains a table of size w , and works as follows: Upon insertion of an item x , if x is in the table, increment its counter by 1. Otherwise, find the item with the smallest counter in the table and replace the key with x . The counter is kept (i.e., it is not reset) and incremented by 1. To look-up the value of item y , traverse the table to find y and output its count. If y is not found in the structure, then its frequency is at most the minimum counter in the table. Furthermore, if the frequency of any item in the stream is larger than the minimum counter in the table, then it must exist in the table [86].

Space Saving in the data plane. Performing this exact algorithm in the data plane is very problematic. Due to the limited number of memory accesses that may be performed while processing a packet, traversing the entire table would require numerous re-circulations and therefore would not be practical. This means that both finding an item x in the table and looking for the minimum counter would not be possible.

Several attempts have been made to adapt this algorithm to the data plane [18, 107]. Hash-Pipe [107] was the first algorithm to adapt the Space Saving algorithm to the data plane. Precision [18] later improved the performance of the algorithm by introducing probabilistic recirculation. Precision is based on a variant of Space Saving called RAP (Random Admission Policy) [19]. As shown in Figure 4, the single key-value table of size w is divided into d hash tables of size w/d each placed in a separate stage. Precision works as follows: When inserting an item x , instead of traversing the entire table to find an item, it will be hashed using d independent hash functions to one index in each of the d tables. If x is found in one of these indexes, then its counter will be incremented by 1. If an item is not found in the structure, then it will be inserted with some probability p . Therefore, not every item seen will necessarily be inserted to the structure. To insert the item, it is recirculated and processed by the pipeline a second time. The probability for insertion (and recirculation) is based on the value of the minimum counter seen in the d indexes, and decreases as the minimum counter increases. While Space Saving sketch may only overestimate the counters, the probabilistic recirculation in Precision introduces a two-sided error, which may either over- or under-estimate the values of the counters.

5 Complex Data Structures

In this section, we discuss approximate data structures that support more sophisticated telemetry queries beyond estimating traffic counts for heavy flows. As described in the previous section, it is possible to make relatively straightforward adjustments to classic data structures such as the count-min sketch so they can function within the data plane. For more sophisticated queries, traditional data structures may perform computations or memory accesses in complex ways that do not have a natural analogue in the data plane; instead, new designs are needed. So far, a wide variety of data structures for the data plane have been introduced to (1) estimate various flow-level statistics over a single flow key definition, such as distinct flows, entropy, and flow changes, (2) answer multiple queries over multiple keys, and (3) compute network performance statistics.

5.1 Sketches for Distinct Counting

Estimating the number of distinct flows is a fundamental problem in network telemetry. Given a definition of a *flow key* (such as a source IP address or 5-tuple), the number of distinct flows is defined as the number of distinct keys appearing in the traffic. To estimate the number of distinct flows, we can consider the classic Linear Counting algorithm [123] as an example. **Linear Counting (LC)** has simple data-plane logic to achieve fast packet processing [128, 130] but cannot maintain high accuracy when the distinct count is large and the memory space is relatively small. At a high level, the LC algorithm needs to maintain a vector of length m and uses a uniformly random hash function to map a flow into an index in the vector (i.e., an m -bit hash table). When all flows are mapped to the hash table, the key idea of LC is to leverage the number of “collisions” happening among m bits, which is an indicator of how many distinct flows are added into the data structure. For example, assuming there are n flows, we consider three possible cases: (1) If $n \ll m$, then the number of bits set to 1 is a good approximation of n . (2) If $n \approx m$, then we need to check how “full” the vector is by counting how many bits are still set to 0. The number of 0s, m_0 , can be used to estimate distinct flows as $\widehat{dist} = m \cdot \ln(\frac{m}{m_0})$. (3) If $n \gg m$, then the approximation will not work well, which is the fundamental limitation for this type of algorithm.

In practice, the LC algorithm is often used in conjunction with the count-min sketch to estimate the number of distinct flows as the LC algorithm is essentially the same as maintaining a row of counters [128]. However, due to the inaccuracy of the LC algorithm in practice when the number of distinct flows is large (e.g., m_0 , as the number of “0” counters, in the sketch becomes small or even 0 as shown in Reference [80]), more accurate distinct counting sketches are needed. Solutions such as LogLog [52] and HyperLogLog [55] (an extension to the Probabilistic Counting algorithm of Flajolet–Martin [56]) are proposed to optimize memory efficiency of counting distinct elements. However, it is challenging to adopt them in the data plane due to a potentially large number of memory accesses. For instance, when adding a new flow into HyperLoglog, we need to perform complex operations on the hash output, such as using only the leftmost subset of the hash index as the counter address and finding the left most “1” in the remaining bits in the hash. Such complex operations manipulating the hash bits are challenging in current programmable data-plane hardware.

Count distinct above the threshold in the data plane: To control the number of memory accesses and avoid complex hash operations in estimating the number of distinct flows, an alternative approach is to estimate whether the distinct count *exceeds a given threshold* (e.g., whether the number of distinct flows is greater than 130). One such method is based on the *coupon collector problem*. At a high level, the coupon-collector problem asks how many random draws (with replacement) are needed to collect all coupons at least once. For instance, we need 129.9 draws in expectation to collect each of 32 coupons. We therefore can use a 32-coupon collector to identify

if the number of distinct flows is at least 130. In a recent effort, BeauCoup [35] explores this idea to build a distinct counting system in the data plane and shows that a coupon drawing process can be implemented efficiently in hardware with one memory access per packet. BeauCoup uses a variant of the coupon-collection scheme that does not assume a uniform probability for collecting all coupons. Instead BeauCoup sets a different probability for drawing each coupon to achieve the desired threshold with a limited number of coupons.

5.2 Sketches for Entropy

In addition to estimating individual flow information, a recent focus has been on estimating metrics that represent entire flow distributions. We call these statistics *distribution measurements*. Compared to estimating individual flows, a distribution measurement requires appropriate metrics to capture and summarize flow attributes of the underlying traffic distribution. Standard statistics in measuring distributions are moments (e.g., standard deviation, mean, kurtosis for tailedness of a distribution, and skewness for the distortion of a distribution), but a more empirically useful statistic for networks is *entropy*, a succinct means of summarizing traffic distributions for anomaly detection [96], DDoS attack detection [82], and fine-grained traffic classification [30]. For instance, a network administrator can track the entropy changes among multiple header fields (e.g., source IP, destination IP, and port number) to identify potential traffic anomalies. A widely adopted definition of entropy is the *Shannon entropy*, defined as $-\sum_{i=1}^n \frac{f_i}{m} \log(\frac{f_i}{m})$, where there are n flows of total size m and each flow i has size f_i .

Lall et al. [96] proposed an entropy estimation algorithm based on the idea of the celebrated AMS sketch [13]. Conceptually, the algorithm can be divided into three steps. In the first step (random selection), the algorithm is prepared to select random locations in the packet stream. These locations decide the set of packets (their flow keys) that the algorithm tracks online. Then in the second step (flow tracking), the algorithm will keep track of the number of packets in particular flows, since the selected packet orders/locations in the stream (i.e., determine which flows are tracked). Finally, in the third step, the entropy can be estimated through tracked flow counters and an offline estimation procedure via logarithmic operations and floating-point number calculations. In summary, this type of sketch algorithm needs to select a randomly chosen set of flows and maintain their flow sizes as the presentation of the entire flow size distribution.

Entropy sketches in the data plane. It is often challenging to implement existing entropy sketches entirely in the data plane. Using the above entropy sketch as an example, its first and second steps can be implemented in the data plane, but the third step cannot. The first step is to randomly select a set of locations in a stream to start tracking flows. Instead of performing uniform sampling on each packet, the control plane needs to pre-compute random samples of locations in the upcoming packet stream and deploy these samples into the programmable data plane to check when packets fly by. Based on these random samples as the locations to start updating the sketch with associated flows, the data plane needs to perform random counter updates as the second step and augment the counters to compute entropy values. However, due to parallel memory accesses and complex entropy estimation operations required in the third step, it cannot be performed in the data plane. Other entropy estimation algorithms, such as References [43, 64], are also infeasible in the data plane. Therefore, a pragmatic solution is to offload the final step of the entropy calculation to the control plane by reporting the sketch counters to the CPU [82]. However, retrieving sketch counters can incur non-trivial delays, due to the inefficient implementation of the control-plane API [91]. A batch-based implementation can speed up counter reading [91]. To further reduce the compute footprint and optimize the latency of such data plane and control plane communication, SketchLib [92] proposes several techniques to reuse hash and counter arrays to

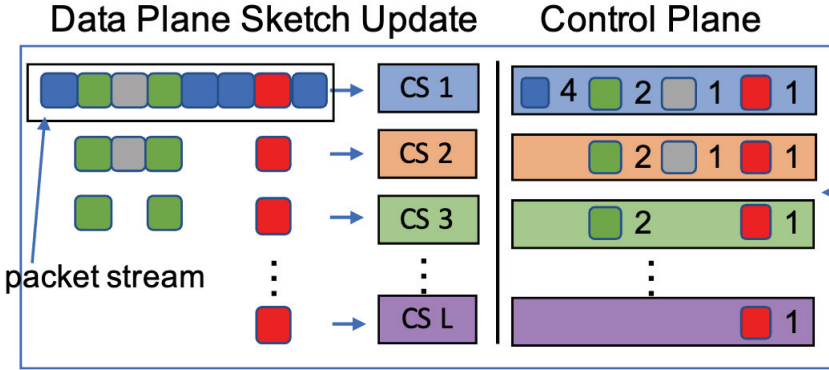


Fig. 5. UnivMon sketch overview.

reduce the counters for entropy calculation. With recent efforts to support floating-point number calculations in the data plane [47, 58, 69, 131], it is a promising direction to design and implement entropy sketches entirely in the data plane to track real-time traffic changes.

5.3 Multi-metric Sketches

In practice, applications today require obtaining traffic metrics based on the keys defined by different tuple fields and their attributes. For example, traffic engineering [16] in the host-level may use the source IP as a key to track heavy hitters, while flow scheduling [102] may need 5-tuple (i.e., protocol, source IP, source port, destination IP, and destination port) as the key. By providing application-specific key definitions, network administrators often need to perform multiple flow-level queries on the *same flow key* or the same query on *distinct flow keys*. For instance, administrators want to query heavy hitters, entropy, and distinct flows over 5-tuple flows using a sketch. Alternatively, for security detection and diagnosis, it is required to query metrics over multiple flow keys and sometimes it is even challenging to predict what specific keys are useful unless we exhaustively track all possible keys. Motivated by these use cases, sketch-based solutions have evolved that can *simultaneously* query *multiple* types of metrics and keys. We categorize these solutions into three categories: “multi-metric, same key,” “single-metric, hierarchical keys,” and “single-metric, separate keys.”

Multiple metrics on the same key. The theoretical foundation of designing a single sketch to estimate multiple statistics comes from the concept of *universal streaming*. The main question that universal streaming seeks to answer is whether such algorithms can be extended to estimate more general metrics of the form $g(f_i)$ for an arbitrary function g defined over the flow distribution. We refer to this statistic as *G-sum* [28]. Conceptually, sketches that can estimate multiple statistics on the same key often consist of multiple single sketches looking at different subsets of the traffic to enable richer queries over the entire traffic. For example, UnivMon [81] uses multiple instances of Count Sketches, called *levels*, to support the estimation of a wide range of traffic statistics, including heavy hitters, distinct flows, entropy, and flow changes. Based on its analysis, the number of levels needed is logarithmically depending the number of distinct flows in the traffic. At a high level, UnivMon leverages theoretical advances in *universal sketching* [27, 28]. When updating the sketch for each packet, UnivMon performs up to $L = O(\log n)$ hashes on the flow key (of the packet) that output a single bit 0 or 1, and starting from the first hash, it uses the longest sequence of 1s to determine if a flow should be tracked at one or multiple levels, as shown in Figure 5. For example, if the first three hashes of a flow key are all 1 but the fourth hash output is 0, then the

hashing process stops and the first three Count Sketch instances will be updated with this flow. This construction of multiple levels of independent sketches is to ensure that the algorithm is able to capture a broad representation of flows for the size estimation across all flows. However, such a packet insertion process can take up to L updates to all the L levels of sketches. Recent efforts of SketchLib [92] and Sketchovsky [93] proposed an efficient insertion operation by only updating the relevant components of the sketch.

Other sketches such as FCM-Sketch [109], PCSA [56], MRAC [72], and multi-resolution bitmap [54] use multiple single arrays of sketch counters. To efficiently implement these multi-query sketches on programmable switches, recently SketchLib [92] has provided a comprehensive library to support the above-mentioned sketches. Recent work, Panakos [137], proposed a sketch that can estimate entropy, distinct count, and tail quantiles accurately, and it is implemented in a P4-based programmable switch. To further improve the accuracy-memory tradeoff, BitSense [50] proposed a compressive sensing framework to compress and recover counters in a sketch.

Single metric over hierarchical keys. A representative traffic metric over hierarchical keys is **hierarchical heavy hitters (HHH)**, where the hierarchy is determined based on the type of prefixes of interest in a given application. The definition of hierarchical keys is motivated by the observation that IP addresses are naturally hierarchical. Suppose that a single entity controls all IP addresses of subnet 66.249.72.*, where * is a wildcard byte. It is possible for the controlling entity to spread out traffic uniformly among this set of IP addresses, so that no single IP address (e.g., 66.249.72.1 and 66.249.72.2) within the set of addresses 66.249.72.* is a heavy hitter, but we may want to know if the sum of the traffic of all IP addresses in the subnet exceeds a specified threshold as a heavy hitter. For instance, anomaly and DDoS detection applications often require identifying frequent flow aggregates based on common IP prefixes, where each device may only generate a small portion of the traffic, but their combined traffic volume is overwhelming.

To answer heavy hitter queries over hierarchical keys, a basic sketch construction for HHH detection is to run an independent sketch instance that detects heavy hitters per layer of the hierarchy. For instance, **Random-HHH (RHHH)** [20] is a sketch that maintains a count-min sketch to track the heavy hitters per layer of the hierarchy. In this way, one can find all possible heavy hitters of all layers (e.g., all prefixes in IP addresses) and thus determine the hierarchical heavy hitters by aggregation. However, updating all sketch instances on all layers is computationally expensive, because there are potentially a large number of layers (e.g., 32 in IPv4). Instead of updating all layers, RHHH randomly selects one level of sketch instances using a level-specific key (e.g., IP prefix) to update per packet. The analysis of RHHH demonstrates that RHHH can achieve similar accuracy as the update-all construction when receiving sufficient packets.

In data plane hardware, maintaining multiple sketch instances for tracking HHHs is resource-heavy and often infeasible when the number of layers is large. CocoSketch [136] tackles this problem by maintaining a single sketch instance and leveraging the hardware resources more efficiently. CocoSketch is designed to support heavy hitter queries over arbitrary flow keys in a pre-defined hierarchy. It is motivated by the challenge to predict what specific keys are relevant before the fact (e.g., security events and performance anomalies). The key insight behind CocoSketch shares the same theoretical basis with Unbiased Space Saving [118] to address the subset sum problem [51]. Given a set of items, each with a weight, the subset sum estimation problem estimates the total weight of any subset of items. The problem of arbitrary partial key queries can be cast as the subset sum estimation problem: the size of a partial-key flow e equals the total size of a subset of full-key flows that match on the partial key with e . For instance, the size of a flow e defined by the fields of source IP and destination IP equals the total size of all 5-tuple flows that share the source IP and destination IP with e . Unbiased Space Saving is

very similar to the original Space-Saving algorithm, yet in the case that the item is not in the table, instead of always replacing the item with the minimum counter, it will only replace it with a probability based on the value of the counter. Using the Unbiased Space Saving technique, CocoSketch minimizes the variance of its subset-sum estimation for querying arbitrary flow keys.

Single metric over separate keys. A straightforward way to estimate a metric over different flow keys would require instantiating multiple separate data structures (e.g., using three count-min sketches to detect the heavy flows in source IP, destination IP, and 5-tuple). Having separate data structures would consume significant memory space in the data plane. What is worse is that to maintain line rate, programmable switches only allow a small constant number of memory accesses per packet, making it infeasible to update multiple data structures for every packet.

A line of recent work [35, 93, 136] has focused on designing a single data structure to estimate a statistic/metric over separate flow keys. BeauCoup adopts the idea of the coupon collector problem to estimate if the number of distinct key values is larger than a threshold for many separate keys (e.g., any packet header fields). For example, BeauCoup can be used to estimate if the number of distinct destination IPs from a source IP is above a certain threshold, which is considered as a “superspreader.” Interestingly, with the ability to measure distinct key values, BeauCoup can also be used to measure heavy hitters by estimating the number of distinct packet IDs in each flow. Specifically, BeauCoup maintains a table with bit vectors representing the coupon collectors. Upon collecting the first coupon for a flow key, BeauCoup creates a new entry in the table. When the bit vector indicates enough coupons are collected, BeauCoup is able to tell if the threshold has been met. Since BeauCoup uses a random mapping from attributes to coupons, observing a new attribute is the same as drawing a coupon and seeing the same attribute more than once does not affect the coupon collector as it is just drawing the same coupon again.

5.4 Performance Statistics

So far, we have discussed data structures to estimate flow-level statistics defined over a stream of individual packets and their aggregated flows. Another important line of telemetry is measuring network performance, since performance problems in the network are notoriously difficult to diagnose. Measuring performance statistics in a resource-efficient way brings new challenges to design and implement data structures for the network data plane.

Compared to measuring individual packets and aggregating the information into flows, monitoring network performance typically requires combining information across pairs of packets in a form of *dependency*: a packet is processed relative to some prior packet of the flow, e.g., we need to measure the time difference between a previous data packet of the flow and the corresponding **acknowledgment (ACK)**. Given such cross-packet dependencies in the measurement, prior work [83] has shown that it is only possible to compute performance metrics over aggregated flow statistics (e.g., total packet loss and latency) using sublinear memory space and it is *infeasible* to compute other performance metrics that rely on one arbitrary pair of packets in a sublinear way, including maximum latency and maximum sending/receiving windows in the flows. The performance metrics on the flows that can be measured in sublinear memory can be called “flow-additive.” This limitation on certain performance metrics motivates the need to design new non-sublinear algorithms that consider the problems of how to store the information about the previous packet and how to avoid bias in the estimation (e.g., bias against the traffic with larger inter-arrival or round-trip latency).

Round-trip time (RTT) is a key indicator of network performance and can often be measured by the difference between the transmission time of the request packet and the receiving time of the corresponding response in a connection/flow. Many latency-sensitive network applications,

such as online gaming or trading, demand fast responses to information about new events, and are therefore extremely sensitive to latency. Hence, minimizing network latency is expected of any adequate network management. To this end, recent efforts [83, 101] have focused on tracking round-trip time in the data plane, because traditional end host-based active probing techniques do not capture application-level RTTs (e.g., sending/receiving new packets isolated from the application) and TCP handshake-based passive monitoring can be inaccurate for long-lived connections.

When measuring RTTs using approximate data structures, such as simple hash tables and sketches, we need to record a request in a flow first and wait for a corresponding response. However, not all requests eventually receive a response, and new requests would suffer from hash collisions with existing requests upon insertion. Upon these collisions, there are two undesirable options: (1) We can discard the new request and keep the existing one, but this will lead to a lot of stale requests staying in the data structure without a response. (2) We can overwrite the existing request with the new one, but requests from flows with larger delays may not survive for a sufficiently long time without collisions, resulting in flows with large delays being undersampled and “bias” toward small-delay flows. Recent work [141] proposed a data structure to correct for this measurement bias in the data plane. Specifically, they track the number of insertions into the data structure for each flow waiting for a response, and they compensate the undersampled flows accordingly when updating the data structure.

Out of order packets are an indicator of the network condition and are often used to infer incorrectly configured **Quality of Service (QoS)**. While there are quite a few root causes of out-of-order packets, faulty QoS configurations, such as setting duplicate Access Control List rules that misclassify the packets from the same application into a different application, potentially delay some packets and fail to keep the packets in order. Such incorrect configurations can affect the performance of online applications. Thus, network flows with many out-of-order packets can be an indicator of current misconfigurations.

Lean algorithms [83], have focused on detecting flows with a high number of out-of-order packets to troubleshoot network configurations using sketches. In this work, the authors provide a definition of characterizing out-of-order packets: Given a stream of packets in one direction, the out-of-order packets are the packets whose sequence numbers are less than the current largest sequence number ($MaxSeq$), i.e., $seq < MaxSeq$, but arrive within a small period of time (e.g., 3 ms) after the packet with $MaxSeq$ is received. Then their objective is to return the k flows with the most out-of-order packets. To track all flows with a high number of out-of-order packets, one needs to compare each incoming packet against the maximum sequence number and latest timestamp of the flow it belongs to. Without knowing this per-flow information, a specific packet cannot be classified as out-of-order and thus the algorithm requires per-flow memory space (non-sublinear). However, Liu et al. [83] obtain a sublinear sketch algorithm to estimate the out-of-order packets per flow with an additional assumption: all out-of-order packets arrive within some bounded time, such as 3ms. If we do not track per-flow out-of-order packets and do not make assumptions about the bounded arrival time, recent work by Zheng et al. [142] shows a sublinear algorithm to track IP prefixes that have high numbers of reordered packets and yields a better memory-accuracy bound than the algorithm in Reference [83].

Time-to-live (TTL) is the amount of “hops” or time in which a packet is set to exist inside a network before being discarded by a switch/router. In the DNS protocol, the TTL value usually specifies how long the corresponding response to a DNS record of a domain should be cached (e.g., 1–3 days). TTL-based features in DNS protocols are particularly useful for finding malicious domains or services. As described in EXPOSURE [23] and Chimera [24], a number of TTL statistics from DNS responses, such as average TTL, standard deviation of TTL, number of distinct TTL

values, and number of TTL changes, are indicators of malicious behaviors. This is because malicious domains and networks often have a sophisticated underlying network infrastructure. This infrastructure can be used to change the routing path by performing attacks such as BGP hijacking, which would normally exhibit TTL changes. Additionally, it is known that a proxy running on a home network would be less reliable (usually with lower TTL values) than a server proxy running on a university environment (with higher TTL values). Given that these TTL-based features can be efficiently measured using sketches described above, sketch-based designs are useful to offer TTL-based performance and security monitoring capabilities.

6 Data-plane Resource Allocation

Deploying compact data structures in network devices requires making efficient use of the limited resources in the data plane to maximize the accuracy of the query results. In this section, we explore ways to quantify measurement accuracy as a function of the size of a compact data structure, and how to optimize data-plane resources to support multiple queries. When the data plane does not have sufficient resources, queries may be partitioned to run partially in the data plane and partially in the control-plane software. The limited data-plane resources are especially challenging to manage for long-running queries, where stale measurement data needs to expire or decay over time.

6.1 Quantifying Measurement Accuracy

The choice of a particular data structure depends on the expected accuracy. Many network applications can tolerate a small number of false positives or slight overestimates of traffic counts. For example, consider a firewall designed to drop unsolicited traffic entering an enterprise. The firewall should admit incoming traffic sent in response to a recent request from an internal host, while dropping other incoming traffic. In this setting, a Bloom filter is an appropriate data structure for storing the set of acceptable flow identifiers (based on the outgoing traffic), because the “no false negatives” property of Bloom filters would ensure that the firewall never drops legitimate response traffic, even if some unsolicited traffic gets through. As another example, consider an application that probabilistically drops packets from large flows to ensure smaller flows get sufficient bandwidth. Errors in estimating the flow counts might lead to slightly higher (or lower) drop rates for some flows, but small errors may be acceptable.

Still, the *amount* of error matters. In the firewall example, admitting a small fraction of unsolicited traffic may be acceptable, but large volumes of unwanted traffic would defeat the purpose of having a firewall. Past theoretical work has led to analytical error bounds for a number of compact data structures [13, 31, 45]. Yet, many of these data structures require modification to work in the data plane—particularly to handle limitations on memory-access bandwidth. Analysis of compact data structures designed for the data plane is an exciting avenue for future research. However, analytical results are often quite conservative. Accuracy is often much higher under realistic traffic than the models would suggest. For example, network traffic often follows a Zipfian distribution, with a small number of large flows and a large number of small flows. Traffic is often bursty, with packets of the same flow arriving close together in time. Incorporating assumptions about the traffic distribution into the analysis could lead to analytical models that provide better estimates of measurement accuracy.

Rather than relying on analytical results, simulations on representative traffic traces can drive the choice of data-structure parameters. Network operators can collect packet traces from their own networks, and use these traces to simulate a candidate data structure and compare the measurement results with ground truth. Using local packet traces has the advantage of capturing a realistic workload for the network in question, though network operators may not know how

long of a trace to use or how diurnal patterns or other traffic shifts might affect the suitability of the data structure. Researchers often use packet traces, too, including publicly available measurement data to better understand how compact data structures perform for realistic traffic across a range of data-structure parameters (e.g., the number of rows r and columns c in a count-min sketch).

An alternative approach is to have the data structures provide estimates of their own measurement error at run time. Run-time reporting of measurement error has the advantage of automatically reacting to changes in the traffic distributions, and providing actionable information to network administrators or the network itself. For example, high measurement error could make a network administrator more conservative in deciding to block or rate limit seemingly suspicious traffic, or could lead to changes in the choice of data structure. Some compact data structures naturally provide a way to quantify their own accuracy. For example, the current values of the $r * c$ counters in a count-min sketch can be used to compute tighter bounds on the estimation error for traffic counts [32]. Designing new kinds of “self-measuring” data structures is an exciting avenue for future research.

6.2 Optimizing Resource Allocation

Deploying compact data structures in practice relies on making decisions about the final size and shape of the structures. Fortunately, approximate data structures are, by their nature, *elastic*; that is, they remain valid under a variety of configurations. For example, for a count-min sketch, increasing the number of rows r or columns c (or both!) increases accuracy, but at the expense of consuming more of the limited data-plane memory. Given resource constraints, a compiler can determine the values of r and c that, together, maximize measurement accuracy, subject to “fitting” within the data-plane target [66, 112]. When the data plane must support multiple telemetry tasks, the compiler can select parameters for each data structure to maximize some weighted objective function that considers the accuracy of each query, subject to all of the data structures fitting within the resource constraints.

Rather than statically allocating a separate data structure for each query, the data plane could use a single shared data structure. This approach has the *statistical multiplexing* advantage of the limited data-plane memory. However, a shared data structure imposes more limits on the number of queries each arriving packet can update. In the simplest case, different queries operate on different traffic, allowing the telemetry system to associate each packet with only one query. More generally, though, queries consider overlapping traffic. For example, one query may identify heavy-hitter source IP addresses, while another identifies heavy-hitter destination IP addresses, and yet another identifies destinations receiving traffic from a large number of distinct sources. Each packet is relevant to all three of these queries. In this setting, sampling can enable each packet to update state for at most one of these three queries, to stay within the data plane’s limited memory access bandwidth [35]. Furthermore, Sketchovsky [93] showed that creating an ensemble of sketches that can allow reuse of hash results in addition to improving memory usage.

Recent work explores a division of time across queries. In one solution, each query is allotted a certain interval of time in which it is performed, according to both the available resources and the accuracy required for the query [88]. In other solutions, queries can be modified on-the-fly, during run-time, enabling the user to actively decide which queries should be performed, according to the changing needs of the network [73, 103, 110, 112, 126, 140].

6.3 Partitioning the Queries

Unfortunately, the data-plane resources may not be sufficient for computing accurate answers for each of the queries. In some cases, the memory size or bandwidth may be too small to handle

every query. In other cases, queries may require complex operations—such as regular expressions on packet payloads, or computing the median of a large set of numbers—that the data plane cannot perform. Rather than sacrificing accuracy or functionality, a telemetry system can *partition* a query to run partially in the data plane and partially in the control plane [61]. For example, a class of Telnet-based attacks on Internet of Things devices involves the adversary sending short Telnet packets with a particular command (e.g., “ZORRO”) in the payload [98]. While parsing strings in the packet payload may be too expensive, the data plane can readily identify small packets using the Telnet transport port, and direct them to control-plane software for further analysis of the packet payload [61].

Alternatively, the telemetry system can capitalize on extra storage space outside of the data plane, to help in accumulating statistics that cannot fit entirely in the data-plane registers [94]. For example, the data plane could store and accumulate statistics for the popular active flows, and maintain information about other flows in a slower memory in the control plane. This approach works well for certain statistics, such as the total number of bytes or packets in a flow. However, merging the aggregate statistics from two locations is not always possible without some loss in accuracy, depending on the statistic of interest. For example, consider a query that counts the number of out-of-order packets in a TCP connection. One way of defining this query is to count the number of packets with sequence numbers smaller than the maximum value seen so far. However, the maximum sequence number may appear very early in a long stream, making it difficult to merge results from the data plane with earlier results stored in the control plane [94].

In these kinds of telemetry systems, a compiler needs to *partition* the set of queries to determine what portion of the analysis can run in the data plane, and what portion must rely on separate processing and memory resources, while minimizing the overhead of involving the control plane. Another way to involve the control-plane software is in analyzing samples or measurements gathered in the data plane. Some recent telemetry systems [17, 76, 81] collect flow samples or flow counters within the programmable data plane and perform further analysis in the control plane to reconstruct the statistics of interest. Despite enabling a richer set of queries, dividing the analysis between the data-plane hardware and the control-plane software often comes with a price. First, the split may incur significant communication overhead. Second, analysis will take orders of magnitude more time than analysis that is performed directly and entirely in the data plane.

6.4 Reusing Data Structures Over Time

The data plane cannot maintain traffic statistics for long intervals of time, since, as the structures fill up, accuracy may be reduced due to excessive collisions. Furthermore, counters could overflow, causing significant errors in counter values.

One way to clear the structure is by decaying old statistics, using methods such as **exponential weighted moving average (EWMA)**. While it is difficult to perform exact EWMA in the data plane, recent work performs approximate EWMA [105]. Another approach is to collect new traffic statistics as time passes by evicting old data [97], or by reinitializing parts of the data structure while continuing to collect new data. For example, the data plane could maintain four data structures, each covering a 10-s period, which will be used as a sliding window. One data structure can be cleaned (i.e., its counters reset to 0) while the other three are used to collect statistics for a 30-s interval. For example, the data structure for times 0–10 could be cleaned during times 30–40, and then populated again during times 40–50 to contribute to the statistics for the intervals 20–50, 30–60, and 40–70, respectively. Sliding windows allow clearing out the data in bulk, while decaying counters using EWMA, for example, allows greater granularity. However, it is harder to implement in the data plane. While a recent work presents a Bloom filter that supports deletion

and decay [48], the topic of decay has not been studied much within the restrictions of the data plane, and remains a promising direction for future work.

Alternatively, instead of using a decaying or sliding window mechanism, a recent solution proposes a new paradigm of *monitoring on demand* [75]. There, the system measures queue-based loss. Such loss can only occur when there is build-up in the queue, and will not happen otherwise. Therefore, the system starts monitoring when required, that is when the queue starts to build up, and stops when the queue winds down. At that point, the structure is cleaned and prepared for the next monitoring interval.

Cleaning data structures. The responsibility for cleaning a data structure can fall to separate control-plane software that accesses the data-plane registers to reinitialize the values. However, the cleaning operations can introduce control-plane overhead and the need for close time synchronization between the control and data planes. An alternative is to “clean” the stale data structure directly from the data plane, as the packets fly by [36]. Each packet, then, would trigger the data plane to clean a portion of one data structure, while reading from the remaining structures and writing to the most recent structure.

During periods of low traffic loads, the data plane may not receive sufficient traffic to complete the cleaning process before the data structure must return to collecting statistics. The switch can ensure sufficient “cleaning traffic” by recirculating a small amount of traffic simply to trigger the data plane to reset the elements of the stale data structure. Another technique to speed up cleaning is based on maintaining a smaller representation of the data structure, where each register is represented by a single bit (i.e., a bitmap). Instead of cleaning the entire data structure, the representation is reset, requiring significantly fewer packets for the process [75]. Then, for each register, the next time it is written to, if its representative bit is still set to 0, the register will first be cleaned, and the bit set to 1, to avoid unnecessarily resetting the register while it is already in use.

7 Distributed Telemetry

In the prior sections we have discussed ways to measure traffic at a single location in the network. However, network administrators often need to collect measurements from multiple vantage points and combine or merge them to get a broader view of the state of the network. In this section, we discuss the challenges of collecting and analyzing measurements from across the network and specifically the need for efficient coordination.

Coordination poses an inherent trade-off; on the one hand, the amount of information collected at each location, as well as the frequency and size of the coordination messages, need to be limited; on the other hand, harsh limitations may degrade the accuracy of the system. We thus need to find ways to optimize this communication. We discuss two main paradigms for performing coordination for distributed telemetry. The first piggybacks information on packets traversing the network; that is, packets ferry state about their own experience or the state of the network. The second makes use of dedicated coordination messages. These paradigms can be used separately or combined, depending on the task at hand.

In this section, we will show how each of these paradigms can be useful for different network tasks and survey existing solutions. Despite promising advances in these areas, the topic remains mostly uncharted, leaving many problems open for further research.

We note that an underlying assumption in our discussion is that measurements may be combined. Looking at basic data structures that we saw in Section 4, certain sketches such as the count-min sketch, are known to be mergeable (assuming they are of the same size and functionality). However, merging other types of data structures (e.g., Space Saving) may be non-trivial. The need for network-wide measurements requires us to find ways to merge these data structures in an efficient manner, yet this will not be the focus of our discussion.

7.1 Network-wide Measurements

In Section 4, we saw data structures for identifying heavy hitters in a single device. Yet, network problems often spread throughout the network. For example, often in DDoS attacks, the attack traffic may not be heavy at a single point, but the sum of the traffic is substantial and constitutes a *network-wide heavy hitter*. To detect the attack in this scenario, measurements performed in various locations in the network need to be aggregated. The aggregation of measurements performed across the network provides the abstraction that the network is a single measurement entity, also known as the *one-big-switch* network abstraction.

One of the key challenges in network-wide monitoring is task placement, that is where in the network should measurements be performed. Systems such as vCRIB [89] provide a solution for source partitioning in host-based network measurements in data centers, that is both resource-aware and traffic-aware. With the rise of network functions and the wide range of programmable devices, the options are vast and there is much opportunity for optimizing task placement in the network [12, 37, 106], as well as more advanced coordination between the measurement points [78]. In addition to placing tasks among switches or hosts, some systems coordinate a network-wide deployment among hosts, switches, and controllers [67, 113].

Another challenge is to determine the rate at which the aggregation is performed. Most existing network-wide telemetry solutions perform measurements (or collect samples) in various locations, which are then reported back to a controller for aggregation [81, 128]. The reporting rate therefore has a significant impact on the accuracy of the aggregated measurements and while higher reporting rates will usually lead to higher accuracy, they may also incur high communication overhead. Many solutions use a fixed reporting rate [17, 49, 62]. An alternate approach seeks to perform continuous monitoring [46, 135], using probabilistic reporting [63].

7.2 Telemetry Across a Path

Monitoring the experience of a packet traversing the network is useful for pinpointing problems and improving performance. Information can be collected about the path that the packet traversed and what the packet experienced along this path. For example, as a packet goes through a switch, it can detect the delay it experienced in the queue by computing the difference between the time it entered the queue and the time it left the queue. As the packet traverses the network, it can aggregate the queuing delay it experienced across the entire path. Path-based queries can include any statistic relevant to a packet, flow or network devices, including queuing loss and flow counts [95, 138] or congestion [60]. Path-based monitoring may even be used for detecting the source of spoofed packets [100, 108]. Data carried by the packets can also be accumulated inside the data plane by the switches along the path of the packet. For example, structures for finding heavy hitters can be used to identify the paths or flows experiencing heavier delays.

The **inband network telemetry (INT)** framework, as found in certain network devices [10], enables telemetry to be collected and/or aggregated within the data plane without the need for controller involvement [71, 116]. In the INT framework, packets may carry measurements and data as well as telemetry instructions that are read and executed by the network devices. The collected measurements are usually sent back to the controller for further analysis, yet it is possible to perform some of the analysis inside the data plane for in-network computation. For example, information on queuing delay can be accumulated in a structure for heavy hitter detection to identify flows that were heavily delayed.

Programmable networks provide the flexibility to collect statistics as defined by the programmer. Yet, if many INT tasks are being performed, then a lot of data may be appended to each packet, thus increasing the load on the network. Yet, programmable devices also enable aggregating

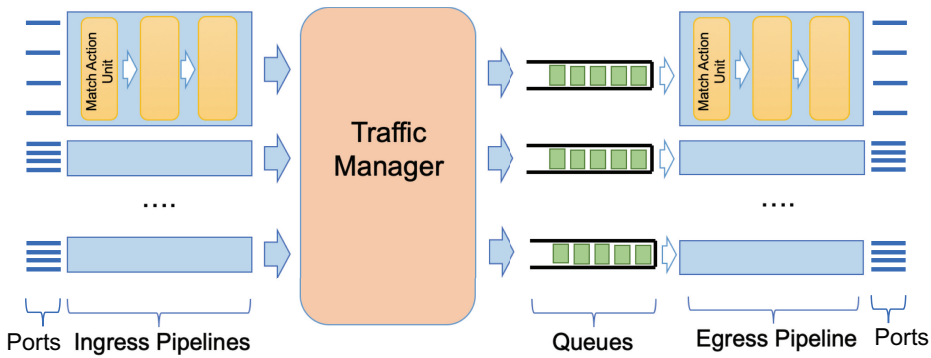


Fig. 6. PISA Programmable Switch Architecture with multiple ingress and egress pipelines. Each port is coupled to a dedicated pipeline. Each packet comes in through an ingress port and traverses the ingress pipeline coupled to this port. The egress port will be determined by the ingress pipeline. The traffic manager receives the packet from the ingress pipeline and forwards it to the queue of the egress pipeline that is coupled to this egress port.

the statistics rather than maintaining per-hop information along the entire path. Furthermore, solutions such as PINT [21], provide a probabilistic variation of INT that bounds the per-packet overhead while providing similar monitoring capabilities.

7.3 Intra-switch Distributed Telemetry

While we often think of distributed telemetry as spanning multiple locations in the network, even collecting telemetry in a single switch is inherently distributed. As seen in Figure 6, programmable switches often have more than one ingress or egress pipeline [124]. Each pipeline serves a given set of ports. Upon entering the switch at an ingress port, a packet is processed by the ingress pipeline that determines which egress port, and respective egress pipeline, should handle the packet as it leaves the switch. Thus, each ingress pipeline potentially feeds all of the egress pipelines.

We consider two main types of intra-switch telemetry. The first measures intra-switch statistics such as loss or delay. Such information may need to be collected across multiple disjoint pipelines within a single switch. While many solutions perform measurements individually for each pipeline [18, 75], in the case of multi-path routing or load-balancing, flows may arrive at different ingress pipelines or exit through different egress pipelines. In this case, collecting any flow-based measurements requires aggregating the counters found in the different pipelines of the switch.

The second type performs joins across pipelines. For example, we might wish to match between a SYN and the respective ACK [82] (or compute the RTT [34, 101] of a flow). Even if both the SYN and ACK packets traverse the same path in the network, they could still be processed by different ingress and/or egress pipelines. Thus, the match or *join* would need to be performed across the different pipelines.

Due to the compartmentalized memory model in programmable switches [9], integrating measurements across pipelines is quite challenging. Certain solutions attempt to overcome this obstacle by ferrying information between ingress and egress on existing traffic [75, 119] or by transferring information between pipelines using designated packets [132]. In these solutions, information passed between pipelines is used to merge or aggregate measurements across pipelines to provide the abstraction of a single structure for all pipelines. Other solutions divide the flow space between pipelines to avoid the need for aggregation [39].

Recently, we have also seen somewhat orthogonal solutions that suggest an extension to the existing data-plane architecture to support stateful packet processing across pipelines [59, 104].

8 Security

In recent years, programmable networks have been used to enhance network security [82, 133, 139], yet apart from a handful of works [14, 57], the *vulnerability* of programmable networks and devices has received less focus. In this section, we explore some of the more common vulnerabilities of programmable devices with respect to network telemetry.

8.1 Better Hash Functions

One of the key components of most measurement data structures in the data plane is the *hash function*. To maintain the processing at line rate, current hardware performs hash functions using hardware implementations. One such common implementation is based on the **cyclic redundancy check (CRC)**. The fixed-length output of the CRC was originally intended to be an error detection code, and is being used in many applications as a hash output. However, CRC is not a cryptographic hash function. The short hash length is vulnerable to collision-based attacks and is therefore considered insecure [121]. Recently, we have seen a solution for implementing a variant of SipHash [15] in the data plane [129], yet the process requires multiple pipeline passes and thus does not keep up with line rate. We have also seen a solution that implements the cryptographic function AES [33] on a programmable switch, yet this solution too requires multiple recirculations to be performed. A faster alternative has been presented using the Even-Mansour scheme that is able to perform packet-header obfuscation at switch hardware rates [120]. These results show that this is a promising direction, yet more robust solutions are still needed.

8.2 Robustness to Adversaries

The data structures themselves can also be vulnerable to attack and adversarial traffic. For example, adversarial traffic may pollute Bloom filters causing a high collision rate and consequently an increase in false positive rate [42], or cause count-min sketches to incorrectly detect large flows [90]. Certain adaptations to these structures have been presented. First, to be more resilient to pollution, the sketches should be as large as possible. For example, the Broom filter [22] divides the filter between local and remote memory, so the overall size of the structure can be much larger. The overhead caused by access to remote memory is reduced due to a unique feedback mechanism between remote and local memory. Second, the standard hardware hash functions are publicly known. Thus, randomness (i.e., a random value or salt) or a secret key should be used when computing hash functions, to prevent an adversary from determining the output of the hash functions and using this information to modify the data [42]. Third, network devices may be vulnerable to various software and hardware bugs, which can allow attackers to gain unauthorized access to sketch data structures and modify the output telemetry results. TrustSketch [38] proposes and optimizes running sketches inside trusted execution environments such as Intel SGX to preserve the integrity of the sketch data structures and the telemetry output.

To enhance the security of network telemetry, and specifically sketch-based measurements, we believe that finding additional vulnerabilities in the structures as well as devising *practical* ways to secure them is a problem that should be further explored.

9 Conclusions

The emergence of high-speed programmable network devices offers the potential for unprecedented visibility into network traffic. Compact data structures are critical for bridging the gap between the questions network administrators want to answer and the limited computation and

memory resources on network devices. The research on compact data structures for network telemetry over the past several years gives a sense of the exciting possibilities, yet so many important research challenges remain. Future research can consider more sophisticated queries (including network performance), as well as better ways to support multiple queries concurrently and to change the collection of queries at run-time. Distributed telemetry and robustness to adversaries are other key directions for further work. Finally, future work can explore how the network devices may take action automatically, to close the control loop. This will allow telemetry systems to go beyond analyzing the traffic to taking actions on traffic directly in the data plane to improve network performance, security, and reliability.

References

- [1] Agilio CX SmartNICs. Retrieved February 16, 2025 from <https://www.netronome.com/products/agilio-cx/>
- [2] Alveo SN1000 SmartNIC: Software defined, hardware accelerated. Retrieved February 16, 2025 from <https://www.xilinx.com/applications/data-center/network-acceleration/alveo-sn1000.html>
- [3] Aruba CX 10000 Switch Series. Retrieved February 16, 2025 from <https://www.arubanetworks.com/products/switches/core-and-data-center/10000-series/>
- [4] Broadcom Trident4. Retrieved February 16, 2025 from <https://www.broadcom.com/products/ethernet-connectivity/switching/strataxgs/bcm56880-series>
- [5] Intel Tofino Series. Retrieved February 16, 2025 from <https://www.intel.com/content/www/us/en/products/network-io/programmable-ethernet-switch.html>
- [6] Mellanox Innova-2 Flex Open Programmable SmartNIC. Retrieved February 16, 2025 from <https://network.nvidia.com/files/doc-2020/pb-innova-2-flex.pdf>
- [7] NPL: Open, High-Level language for developing feature-rich solutions for programmable networking platforms. Retrieved February 16, 2025 from <https://nplang.org/>
- [8] Pensando DSC-200 Distributed Services Card. Retrieved February 16, 2025 from <https://www.amd.com/system/files/documents/pensando-dsc-200-product-brief.pdf>
- [9] P416 Intel Tofino Native Architecture – Public Version. Retrieved February 16, 2025 from https://github.com/barefootnetworks/OpenTofino/blob/master/PUBLIC_Tofino-Native-Arch.pdf
- [10] Intel Deep Insight Network Analytics Software. Retrieved February 16, 2025 from <https://www.intel.com/content/www/us/en/products/network-io/programmable-ethernet-switch/network-analytics/deep-insight.html>
- [11] Announcing Next-Generation P4-Programmable Datacenter Switching. Retrieved February 16, 2025 from <https://www.linkedin.com/pulse/announcing-next-generation-p4-programmable-datacenter-switching-szkoc/>
- [12] Anup Agarwal, Zaoxing Liu, and Srinivasan Seshan. 2022. HeteroSketch: Coordinating network-wide monitoring in heterogeneous and dynamic networks. In *Proceedings of the USENIX Symposium on Networked Systems Design and Implementation*. USENIX Association, 719–741.
- [13] Noga Alon, Yossi Matias, and Mario Szegedy. 1996. The space complexity of approximating the frequency moments. In *Proceedings of the ACM Symposium on Theory of Computing*. ACM, New York, NY, 20–29. DOI: <https://doi.org/10.1145/237814.237823>
- [14] Ali AlSabe, Joseph Khoury, Elie Kfoury, Jorge Crichigno, and Elias Bou-Harb. 2022. A survey on security applications of P4 programmable switches and a STRIDE-based vulnerability assessment. *Comput. Netw.* 207 (2022), 108800.
- [15] Jean-Philippe Aumasson and Daniel J. Bernstein. 2012. SipHash: A fast short-input PRF. In *Proceedings of the Conference on Progress in Cryptology (INDOCRYPT'12)*. 489–508.
- [16] D. Awduche, A. Chiu, A. Elwalid, I. Widjaja, and X. Xiao. 2002. Overview and Principles of Internet Traffic Engineering. Retrieved from <https://tools.ietf.org/html/rfc3272>
- [17] Ran Ben-Basat, Xiaoqi Chen, Gil Einziger, Shir Landau Feibish, Danny Raz, and Minlan Yu. 2020. Routing oblivious measurement analytics. In *Proceedings of the IFIP Networking Conference*. 449–457.
- [18] Ran Ben-Basat, Xiaoqi Chen, Gil Einziger, and Ori Rottenstreich. 2020. Designing heavy-hitter detection algorithms for programmable switches. *IEEE/ACM Trans. Netw.* 28, 3 (2020), 1172–1185.
- [19] Ran Ben-Basat, Gil Einziger, Roy Friedman, and Yaron Kassner. 2017. Randomized admission policy for efficient top-k and frequency estimation. In *Proceedings of the IEEE INFOCOM*.
- [20] Ran Ben-Basat, Gil Einziger, Roy Friedman, Marcelo Caggiani Luizelli, and Erez Waisbard. 2017. Constant time updates in hierarchical heavy hitters. In *Proceedings of the ACM SIGCOMM*.
- [21] Ran Ben-Basat, Sivaramakrishnan Ramanathan, Yuliang Li, Gianni Antichi, Minian Yu, and Michael Mitzenmacher. 2020. PINT: Probabilistic in-band network telemetry. In *Proceedings of the ACM SIGCOMM*.

- [22] Michael A. Bender, Martin Farach-Colton, Mayank Goswami, Rob Johnson, Samuel McCauley, and Shikha Singh. 2018. Bloom filters, adaptivity, and the dictionary problem. In *Proceedings of the IEEE Annual Symposium on Foundations of Computer Science*.
- [23] Leyla Bilge, Engin Kirda, Christopher Kruegel, and Marco Balduzzi. 2011. Exposure: Finding malicious domains using passive DNS analysis. In *Proceedings of the Network and Distributed System Security Symposium (NDSS'11)*. 1–17.
- [24] Kevin Borders, Jonathan Springer, and Matthew Burnside. 2012. Chimera: A declarative language for streaming network traffic analysis. In *Proceedings of the 21st USENIX Security Symposium (USENIX Security'12)*. 365–379.
- [25] Pat Bosshart, Dan Daly, Glen Gibb, Martin Izzard, Nick McKeown, Jennifer Rexford, Cole Schlesinger, Dan Talayco, Amin Vahdat, George Varghese, and David Walker. 2014. P4: Programming protocol-independent packet processors. *ACM SIGCOMM Comput. Commun. Review* 44, 3 (2014), 87–95.
- [26] Pat Bosshart, Glen Gibb, Hun-Seok Kim, George Varghese, Nick McKeown, Martin Izzard, Fernando Mujica, and Mark Horowitz. 2013. Forwarding metamorphosis: Fast programmable match-action processing in hardware for SDN. In *Proceedings of the ACM SIGCOMM*.
- [27] Vladimir Braverman, Robert Krauthgamer, and Lin F. Yang. 2018. Universal streaming of subset norms. Retrieved from <https://arxiv.org/abs/1812.00241>
- [28] Vladimir Braverman and Rafail Ostrovsky. 2010. Zero-one frequency laws. In *Proceedings of the ACM Symposium on Theory of Computing*.
- [29] Andrei Broder and Michael Mitzenmacher. 2002. Network applications of Bloom filters: A survey. In *Internet Mathematics* 1, 4 (2002), 485–509.
- [30] Amit Chakrabarti, Graham Cormode, and Andrew McGregor. 2010. A near-optimal algorithm for estimating the entropy of a stream. *ACM Trans. Algorithms* 6, 3 (2010), 1–21.
- [31] Moses Charikar, Kevin Chen, and Martin Farach-Colton. 2004. Finding frequent items in data streams. *Theor. Comput. Sci.* 312, 1 (Jan. 2004), 3–15.
- [32] Peiqing Chen, Yuhan Wu, Tong Yang, Junchen Jiang, and Zaoxing Liu. 2021. Precise error estimation for sketch-based flow measurement. In *Proceedings of the ACM SIGCOMM Internet Measurement Conference*.
- [33] Xiaoqi Chen. 2020. Implementing AES encryption on programmable switches via scrambled lookup tables. In *Proceedings of the ACM SIGCOMM Workshop on Secure Programmable Network Infrastructure*. ACM, 8–14.
- [34] Xiaoqi Chen, Hyojoon Kim, Javed M. Aman, Willie Chang, Mack Lee, and Jennifer Rexford. 2020. Measuring TCP round-trip time in the data plane. In *Proceedings of the ACM SIGCOMM Workshop on Secure Programmable Network Infrastructure (SPIN'20)*. 35–41.
- [35] Xiaoqi Chen, Shir Landau Feibish, Mark Braverman, and Jennifer Rexford. 2020. BeauCoup: Answering many network traffic queries, one memory update at a time. In *Proceedings of the ACM SIGCOMM*.
- [36] Xiaoqi Chen, Shir Landau Feibish, Yaron Koral, Jennifer Rexford, Ori Rottenstreich, Steven A. Monetti, and Tzuu-Yi Wang. 2019. Fine-grained queue measurement in the data plane. In *Proceedings of the ACM SIGCOMM Conference on Emerging Networking Experiments and Technologies*. ACM, 15–29.
- [37] Xiang Chen, Qingjiang Xiao, Hongyan Liu, Qun Huang, Dong Zhang, Xuan Liu, Longbing Hu, Haifeng Zhou, Chunming Wu, and Kui Ren. 2024. Eagle: Toward scalable and near-optimal network-wide sketch deployment in network measurement. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 291–310.
- [38] Zhuo Cheng, Maria Apostolaki, Zaoxing Liu, and Vyas Sekar. 2024. TrustSketch: Trustworthy sketch-based telemetry on cloud hosts. In *Proceedings of the Network and Distributed System Security Symposium (NDSS'24)*.
- [39] Marco Chiesa and Fábio L. Verdi. 2023. Network monitoring on multi-pipe switches. In *Proceedings of the ACM SIGMETRICS*. ACM. <https://doi.org/10.1145/3578338.3593554>
- [40] Benoit Claise. 2004. Cisco systems netflow services export version 9. RFC 3954. Retrieved from <https://datatracker.ietf.org/doc/html/rfc3954>
- [41] B. Claise, B. Trammell, and P. Aitken. 2013. Specification of the IP Flow Information Export (IPFIX) Protocol for the Exchange of Flow Information. RFC 7011. Retrieved from <https://datatracker.ietf.org/doc/html/rfc7011>.
- [42] David Clayton, Christopher Patton, and Thomas Shrimpton. 2019. Probabilistic data structures in adversarial environments. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*. ACM, 1317–1334.
- [43] Peter Clifford and Ioana Cosma. 2013. A simple sketching algorithm for entropy estimation over streaming data. In *Proceedings of the International Conference on Artificial Intelligence and Statistics*.
- [44] Graham Cormode. 2011. Sketch techniques for approximate query processing. *Found. Trends Databases* 15 (2011).
- [45] Graham Cormode and Shan Muthukrishnan. 2005. An improved data stream summary: The count-min sketch and its applications. *J. Algor.* 55, 1 (2005), 58–75.
- [46] Graham Cormode, S. Muthukrishnan, and Ke Yi. 2011. Algorithms for distributed functional monitoring. *ACM Trans. Algor.* 7, 2 (2011), 21:1–21:20. DOI : <https://doi.org/10.1145/1921659.1921667>
- [47] Penglai Cui, Heng Pan, Zhenyu Li, Jiaoren Wu, Shengzhuo Zhang, Xingwu Yang, Hongtao Guan, and Gaogang Xie. 2021. NetFC: Enabling accurate floating-point arithmetic on programmable switches. In *Proceedings of the IEEE International Conference on Network Protocols (ICNP'21)*. IEEE, 1–11.

- [48] Jonathan Diamant and Shir Landau Feibish. 2024. SetD4: Sets with deletions and decay in the data plane. *Proc. ACM Netw.* 2, CoNEXT4, Article 34 (Nov. 2024), 22 pages. DOI : <https://doi.org/10.1145/3696391>
- [49] Damu Ding, Marco Savi, Gianni Antichi, and Domenico Siracusa. 2019. Incremental deployment of programmable switches for network-wide heavy-hitter detection. In *Proceedings of the IEEE Conference on Network Softwarization (NetSoft'19)*. 160–168. DOI : <https://doi.org/10.1109/NETSOFT.2019.8806649>
- [50] Rui Ding, Shibo Yang, Xiang Chen, and Qun Huang. 2023. Bitsense: Universal and nearly zero-error optimization for sketch counters with compressive sensing. In *Proceedings of the ACM SIGCOMM Conference*. 220–238.
- [51] Nick G. Duffield, Carsten Lund, and Mikkel Thorup. 2007. Priority sampling for estimation of arbitrary subset sums. *J. ACM* 54, 6 (2007), 32.
- [52] Marianne Durand and Philippe Flajolet. 2003. Loglog counting of large cardinalities (extended abstract). In *Proceedings of the 11th Annual European Symposium on Algorithms (ESA'03) (Lecture Notes in Computer Science, Vol. 2832)*. Springer, 605–617.
- [53] Cristian Estan and George Varghese. 2002. New directions in traffic measurement and accounting. In *Proceedings of the ACM SIGCOMM*.
- [54] Cristian Estan, George Varghese, and Mike Fisk. 2003. Bitmap algorithms for counting active flows on high speed links. In *Proceedings of the ACM SIGCOMM Internet Measurement Conference*. 153–166.
- [55] Philippe Flajolet, Eric Fusy, Olivier Gandouet, and Frederic Meunier. 2007. Hyperloglog: The analysis of a near-optimal cardinality estimation algorithm. In *Proceedings of the International Conference on Probabilistic, Combinatorial and Asymptotic Methods for the Analysis of Algorithms*.
- [56] Philippe Flajolet and G. Nigel Martin. 1985. Probabilistic counting algorithms for data base applications. *J. Comput. Syst. Sci.* 31, 2 (1985), 182–209.
- [57] Lucas Freire, Miguel C. Neves, Alberto E. Schaeffer Filho, and Marinho P. Barcellos. 2017. POSTER: Finding vulnerabilities in P4 programs with assertion-based verification. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*. ACM, 2495–2497.
- [58] Sam Gao, Mark Handley, and Stefano Vissicchio. 2021. Stats 101 in P4: Towards in-switch anomaly detection. In *Proceedings of the 20th ACM Workshop on Hot Topics in Networks (HotNets'21)*. New York, NY, 84–90. <https://doi.org/10.1145/3484266.3487370>
- [59] Nadeen Gebara, Alberto Lerner, Mingran Yang, Minlan Yu, Paolo Costa, and Manya Ghobadi. 2020. Challenging the stateless quo of programmable switches. In *Proceedings of the ACM Workshop on Hot Topics in Networks (HotNets'20)*. 7 pages.
- [60] Matthias Grossglauser and Jennifer Rexford. 2005. Passive traffic measurement for internet protocol operations. In *The Internet as a Large-Scale Complex System*, Kihong Park and Walter Willinger (Eds.). Oxford University Press, 91–120.
- [61] Arpit Gupta, Rob Harrison, Marco Canini, Nick Feamster, Jennifer Rexford, and Walter Willinger. 2018. Sonata: Query-driven streaming network telemetry. In *Proceedings of the ACM SIGCOMM*.
- [62] Rob Harrison, Qizhe Cai, Arpit Gupta, and Jennifer Rexford. 2018. Network-wide heavy hitter detection with commodity switches. In *Proceedings of the ACM SIGCOMM Symposium on SDN Research*.
- [63] Rob Harrison, Shir Landau Feibish, Arpit Gupta, Ross Teixeira, S. Muthukrishnan, and Jennifer Rexford. 2020. Carpe elephants: Seize the global heavy hitters. In *Proceedings of the ACM SIGCOMM Workshop on Secure Programmable Network Infrastructure*. 15–21.
- [64] Nicholas J. A. Harvey, Jelani Nelson, and Krzysztof Onak. 2008. Sketching and streaming entropy via approximation theory. In *Proceedings of the IEEE Symposium on Foundations of Computer Science*. IEEE, 489–498.
- [65] Frederik Hauser, Marco Häberle, Daniel Merling, Steffen Lindner, Vladimir Gurevich, Florian Zeiger, Reinhard Frank, and Michael Menth. 2023. A survey on data plane programming with P4: Fundamentals, advances, and applied research. *J. Netw. Comput. Appl.* 212 (2023), 103561. DOI : <https://doi.org/10.1016/j.jnca.2022.103561>
- [66] Mary Hogan, Shir Landau Feibish, Mina Tahmasbi Arashloo, Jennifer Rexford, and David Walker. 2022. Modular switch programming under resource constraints. In *Proceedings of the USENIX Networked Systems Design and Implementation*.
- [67] Qun Huang, Haifeng Sun, Patrick P. C. Lee, Wei Bai, Feng Zhu, and Yungang Bao. 2020. Omnimon: Re-architecting network telemetry with resource efficiency and full accuracy. In *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication*. 404–421.
- [68] Nikita Ivkin, Zhuolong Yu, Vladimir Braverman, and Xin Jin. 2019. QPipe: Quantiles sketch fully in the data plane. In *Proceedings of the 15th International Conference on Emerging Networking Experiments And Technologies (CoNEXT'19)*. 285–291. <https://doi.org/10.1145/3359989.3365433>
- [69] Matthews Jose, Kahina Lazri, Jérôme François, and Olivier Festor. 2021. InREC: In-network REal number computation. In *Proceedings of the 17th IFIP/IEEE International Symposium on Integrated Network Management (IM'21)*. IEEE, 358–366.

- [70] Elie F. Kfoury, Jorge Crichigno, and Elias Bou-Harb. 2021. An exhaustive survey on P4 programmable data plane switches: Taxonomy, applications, challenges, and future trends. *IEEE Access* 9 (2021), 87094–87155.
- [71] Changhoon Kim, Anirudh Sivaraman, Naga Katta, Antonin Bas, Advait Dixit, and Lawrence J. Wobker. 2015. In-band network telemetry via programmable dataplanes. In *Proceedings of the Demo Session of ACM SIGCOMM*.
- [72] Abhishek Kumar, Minh Sung, Jun Xu, and Jia Wang. 2004. Data streaming algorithms for efficient and accurate estimation of flow size distribution. *ACM SIGMETRICS Perform. Eval. Rev.* 32, 1 (2004), 177–188.
- [73] Paolo Laffranchini, Luis Rodrigues, Marco Canini, and Balachander Krishnamurthy. 2019. Measurements as first-class artifacts. In *Proceedings of the IEEE Conference on Computer Communications (INFOCOM'19)*. IEEE, 415–423.
- [74] Ashwin Lal, Vyas Sekar, Mitsunori Ogihara, Jun (Jim) Xu, and Hui Zhang. 2006. Data streaming algorithms for estimating entropy of network traffic. In *Proceedings of the Joint International Conference on Measurement and Modeling of Computer Systems (SIGMETRICS/Performance'06)*. ACM, 145–156. <https://doi.org/10.1145/1140277.1140295>
- [75] Shir Landau Feibish, Zaoxing Liu, Nikita Ivkin, Xiaoqi Chen, Vladimir Braverman, and Jennifer Rexford. 2022. Flow-level loss detection with Δ -sketches. In *Proceedings of the ACM Symposium on SDN Research*. ACM, 25–32.
- [76] Yuliang Li, Rui Miao, Changhoon Kim, and Minlan Yu. 2016. FlowRadar: A better netflow for data centers. In *Proceedings of the USENIX Conference on Networked Systems Design and Implementation*. 311–324.
- [77] Yuliang Li, Rui Miao, Hongqiang Harry Liu, Yan Zhuang, Fei Feng, Lingbo Tang, Zheng Cao, Ming Zhang, Frank Kelly, Mohammad Alizadeh et al. 2019. HPCC: High precision congestion control. In *Proceedings of the ACM SIGCOMM*. 44–58.
- [78] Xuemei Liu, Meral Shirazipour, Minlan Yu, and Ying Zhang. 2016. MOZART: Temporal coordination of measurement. In *Proceedings of the ACM SIGCOMM Symposium on SDN Research*. New York, NY.
- [79] Zaoxing Liu, Zhihao Bai, Zhenming Liu, Xiaozhou Li, Changhoon Kim, Vladimir Braverman, Xin Jin, and Ion Stoica. 2019. DistCache: Provable load balancing for large-scale storage systems with distributed caching. In *Proceedings of the USENIX Conference on File and Storage Technologies*.
- [80] Zaoxing Liu, Ran Ben-Basat, Gil Einziger, Yaron Kassner, Vladimir Braverman, Roy Friedman, and Vyas Sekar. 2019. Nitrosketch: Robust and general sketch-based monitoring in software switches. In *Proceedings of the ACM SIGCOMM*.
- [81] Zaoxing Liu, Antonis Manousis, Gregory Vorsanger, Vyas Sekar, and Vladimir Braverman. 2016. One sketch to rule them all: Rethinking network flow monitoring with UnivMon. In *Proceedings of the ACM SIGCOMM*.
- [82] Zaoxing Liu, Hun Namkung, Georgios Nikolaidis, Jeongkeun Lee, Changhoon Kim, Xin Jin, Vladimir Braverman, Minlan Yu, and Vyas Sekar. 2021. Jaqen: A high-performance switch-native approach for detecting and mitigating volumetric DDoS attacks with programmable switches. In *Proceedings of the USENIX Security Symposium*. 3829–3846.
- [83] Zaoxing Liu, Samson Zhou, Ori Rottenstreich, Vladimir Braverman, and Jennifer Rexford. 2020. Memory-efficient performance monitoring on programmable switches with lean algorithms. In *Proceedings of the Symposium on Algorithmic Principles of Computer Systems*. 31–44.
- [84] Boon Thau Loo, Rajeev Alur, Sajal Marwaha, Ankit Mishra, Dong Lin, and Yifei Yuan. 2017. Quantitative network monitoring with NetQRE. In *Proceedings of the ACM SIGCOMM*. 99–112.
- [85] Pilar Manzanares-Lopez, Juan Pedro Muñoz-Gea, and Josemaria Malgosa-Sanahuja. 2021. Passive in-band network telemetry systems: The potential of programmable data plane on network-wide telemetry. *IEEE Access* 9 (2021), 20391–20409.
- [86] Ahmed Metwally, Divyakant Agrawal, and Amr El Abbadi. 2005. Efficient computation of frequent and top-k elements in data streams. In *Proceedings of the International Conference on Database Theory*.
- [87] Oliver Michel, Roberto Bifulco, Gábor Rétvári, and Stefan Schmid. 2021. The programmable data plane: Abstractions, architectures, algorithms, and applications. *ACM Comput. Surveys* 54, 4, Article 82 (May 2021), 36 pages.
- [88] Chris Misa, Walt O'Connor, Ramakrishnan Durairajan, Reza Rejaie, and Walter Willinger. 2022. Dynamic scheduling of approximate telemetry queries. In *Proceedings of the USENIX Symposium on Networked Systems Design and Implementation*. USENIX Association, Renton, WA, 701–717. Retrieved from <https://www.usenix.org/conference/nsdi22/presentation/misa>
- [89] Masoud Moshref, Minlan Yu, Abhishek B. Sharma, and Ramesh Govindan. 2013. Scalable rule management for data centers. In *Proceedings of the USENIX Symposium on Networked Systems Design and Implementation*. USENIX Association, 157–170. Retrieved from <https://www.usenix.org/conference/nsdi13/technical-sessions/presentation/moshref>
- [90] Josu Murua and Pedro Reviriego. 2020. Faking elephant flows on the count min sketch. *IEEE Netw. Lett.* 2, 4 (2020), 199–202. DOI: <https://doi.org/10.1109/LNET.2020.3035272>
- [91] Hun Namkung, Daehyeok Kim, Zaoxing Liu, Vyas Sekar, and Peter Steenkiste. 2021. Telemetry retrieval inaccuracy in programmable switches: Analysis and recommendations. In *Proceedings of the ACM SIGCOMM Symposium on SDN Research (SOSR'21)*. 176–182.
- [92] Hun Namkung, Zaoxing Liu, Daehyeok Kim, Vyas Sekar, Peter Steenkiste, Guyue Liu, Ao Li, Christopher Canel, Adithya Abraham Philip, Ranysha Ware et al. 2022. Sketchlib: Enabling efficient sketch-based monitoring on programmable switches. In *Proceedings of the USENIX Conference on Networked Systems Design and Implementation*.

- [93] Hun Namkung, Zaoxing Liu, Daehyeok Kim, Vyas Sekar, Peter Steenkiste, Guyue Liu, Ao Li, Christopher Canel, Adithya Abraham Philip, Ranysha Ware et al. 2023. Sketchovsky: Enabling ensembles of sketches on programmable switches. In *Proceedings of the USENIX Conference on Networked Systems Design and Implementation*.
- [94] Srinivas Narayana, Anirudh Sivaraman, Vikram Nathan, Prateesh Goyal, Venkat Arun, Mohammad Alizadeh, Vimalakumar Jeyakumar, and Changhoon Kim. 2017. Language-directed hardware design for network performance monitoring. In *Proceedings of the ACM SIGCOMM*. 85–98.
- [95] Srinivas Narayana, Mina Tahmasbi, Jennifer Rexford, and David Walker. 2016. Compiling path queries. In *Proceedings of the USENIX Symposium on Networked Systems Design and Implementation (NSDI'16)*.
- [96] George Nychis, Vyas Sekar, David G. Andersen, Hyong Kim, and Hui Zhang. 2008. An empirical evaluation of entropy-based traffic anomaly detection. In *Proceedings of the ACM SIGCOMM Internet Measurement Conference*.
- [97] Aviya Ozery, Jonathan Diamant, and Shir Landau Feibish. 2024. RecenTo: Finding top-k flows of the recent past. *Proc. ACM Netw.* 2, CoNEXT3 (2024), 1–20. DOI: <https://doi.org/10.1145/3676871>
- [98] Yin Minn Pa Pa, Shogo Suzuki, Katsunari Yoshioka, Tsutomu Matsumoto, Takahiro Kasama, and Christian Rossow. 2015. IoT POT: Analysing the rise of IoT compromises. In *Proceedings of the USENIX Workshop on Offensive Technologies*.
- [99] J. Quittek, T. Zseby, B. Claise, and S. Zander. 2004. Requirements for IP Flow Information Export (IPFIX). RFC 3917. Retrieved from <https://datatracker.ietf.org/doc/html/rfc3917>
- [100] Stefan Savage, David Wetherall, Anna R. Karlin, and Thomas E. Anderson. 2000. Practical network support for IP traceback. In *Proceedings of the ACM SIGCOMM*. ACM, 295–306.
- [101] Satadal Sengupta, Hyojoon Kim, and Jennifer Rexford. 2022. Continuous in-network round-trip time monitoring. In *Proceedings of the ACM SIGCOMM*. 473–485.
- [102] Naveen Kr. Sharma, Ming Liu, Kishore Atreya, and Arvind Krishnamurthy. 2018. Approximating fair queueing on reconfigurable switches. In *Proceedings of the USENIX Conference on Networked Systems Design and Implementation*. USENIX Association.
- [103] Chaofan Shou, Rohan Bhatia, Arpit Gupta, Rob Harrison, Daniel Lokshtanov, and Walter Willinger. 2024. Query planning for robust and scalable hybrid network telemetry systems. *Proc. ACM Netw.* 2, CoNEXT1, Article 3 (Mar. 2024), 27 pages. DOI: <https://doi.org/10.1145/3649471>
- [104] Vishal Shrivastav. 2022. Stateful multi-pipelined programmable switches. In *Proceedings of the ACM SIGCOMM*.
- [105] Suneet Kumar Singh, Christian Esteve Rothenberg, Marcelo Caggiani Luizelli, Gianni Antichi, Pedro Henrique Gomes, and Gergely Pongrácz. 2023. HH-IPG: Leveraging inter-packet gap metrics in P4 hardware for heavy hitter detection. *IEEE Trans. Netw. Service Manage.* 20, 3 (2023), 3536–3548.
- [106] Anirudh Sivaraman, Thomas Mason, Aurojit Panda, Ravi Netravali, and Sai Anirudh-Kondaveeti. 2020. Network architecture in the age of programmability. *ACM SIGCOMM Comput. Commun. Rev.* 50, 1 (2020), 38–44. DOI: <https://doi.org/10.1145/3390251.3390257>
- [107] Vibhaalakshmi Sivaraman, Srinivas Narayana, Ori Rottenstreich, S. Muthukrishnan, and Jennifer Rexford. 2017. Heavy-hitter detection entirely in the data plane. In *Proceedings of the ACM Symposium on SDN Research*.
- [108] Alex C. Snoeren, Craig Partridge, Luis A. Sanchez, Christine E. Jones, Fabrice Tchakountio, Stephen T. Kent, and W. Timothy Strayer. 2001. Hash-based IP traceback. In *Proceedings of the ACM SIGCOMM*. ACM, 3–14.
- [109] Cha Hwan Song, Pravein Govindan Kannan, Bryan Kian Hsiang Low, and Mun Choon Chan. 2020. Fcm-sketch: Generic network measurements with data plane support. In *Proceedings of the ACM SIGCOMM CoNEXT Conference*. 78–92.
- [110] Milind Srivastava, Shao-Tse Hung, Hun Namkung, Kate Ching-Ju Lin, Zaoxing Liu, and Vyas Sekar. 2024. Raising the level of abstraction for sketch-based network telemetry with sketchplan. In *Proceedings of the ACM Conference on Internet Measurement*. 651–658.
- [111] Sari Sultan, Kia Shakiba, Albert Lee, Paul Chen, and Michael Stumm. 2024. TTLs matter: Efficient cache sizing with TTL-aware miss ratio curves and working set sizes. In *Proceedings of the 19th European Conference on Computer Systems (EuroSys'24)*. 387–404. <https://doi.org/10.1145/3627703.3650066>
- [112] Haifeng Sun, Qun Huang, Jinbo Sun, Wei Wang, Jiaheng Li, Fuliang Li, Yungang Bao, Xin Yao, and Gong Zhang. 2024. AutoSketch: Automatic sketch-oriented compiler for query-driven network telemetry. In *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI'24)*. 1551–1572.
- [113] Haifeng Sun, Jiaheng Li, Jintao He, Jie Gui, and Qun Huang. 2023. OmniWindow: A general and efficient window mechanism framework for network telemetry. In *Proceedings of the ACM SIGCOMM Conference*. 867–880.
- [114] Yu-Wei Eric Sung, Xiaozheng Tie, Starsky H. Y. Wong, and Hongyi Zeng. 2016. Robotron: Top-down network management at facebook scale. In *Proceedings of the ACM SIGCOMM Conference*. 426–439.
- [115] Lizhuang Tan, Wei Su, Wei Zhang, Jianhui Lv, Zhenyi Zhang, Jingying Miao, Xiaoxi Liu, and Na Li. 2021. In-band network telemetry: A survey. *Comput. Netw.* 186 (2021), 107763. DOI: <https://doi.org/10.1016/j.comnet.2020.107763>

- [116] The P4 Applications Working Group. 2020. In-band network telemetry (INT) dataplane specification version 2.1. Retrieved from https://p4.org/p4-spec/docs/INT_v2_1.pdf
- [117] The P4 Language Consortium. 2023. P4₁₆ Language Specifications. Retrieved from <https://p4.org/p4-spec/docs/P4-16-v1.2.4.html>
- [118] Daniel Ting. 2018. Data sketches for disaggregated subset sum and frequent item estimation. In *Proceedings of the International Conference on Management of Data*. 1129–1140.
- [119] Fábio Luciano Verdi and Marco Chiesa. 2021. Heavy hitter detection on multi-pipeline switches. In *Proceedings of the ACM Symposium on Architectures for Networking and Communications Systems*. ACM, 121–124.
- [120] Liang Wang, Hyojoon Kim, Prateek Mittal, and Jennifer Rexford. 2021. Programmable in-network obfuscation of DNS traffic. In *Proceedings of the NDSS DNS Privacy Workshop*.
- [121] Liang Wang, Prateek Mittal, and Jennifer Rexford. 2022. Data-plane security applications in adversarial settings. *Proceedings of the ACM SIGCOMM Computer Communications Review*.
- [122] Mea Wang, Baochun Li, and Zongpeng Li. 2004. sFlow: Towards resource-efficient and agile service federation in service overlay networks. In *Proceedings of the IEEE International Conference on Distributed Computing Systems*. 628–635.
- [123] Kyu-Young Whang, Brad T. Vander-Zanden, and Howard M. Taylor. 1990. A linear-time probabilistic counting algorithm for database applications. *ACM Trans. Database Syst.* 15, 2 (1990), 208–229.
- [124] Bob Wheeler. 2019. Tomahawk 4 Switch First to 25.6tbps broadcom doubles 400gbps ports with unprecedented 512 serdes. Retrieved from <https://docs.broadcom.com/doc/12398014>
- [125] Jiarong Xing, Qiao Kang, and Ang Chen. 2020. NetWarden: Mitigating network covert channels while preserving performance. In *Proceedings of the USENIX Security Symposium*. 2039–2056.
- [126] Kaicheng Yang, Yuhuan Wu, Ruijie Miao, Tong Yang, Zirui Liu, Zicang Xu, Rui Qiu, Yikai Zhao, Hanglong Lv, Zhigang Ji, and Gaogang Xie. 2023. Chameleon: Shifting measurement attention as network state changes. In *Proceedings of the ACM SIGCOMM*.
- [127] Mingran Yang, Alex Baban, Valery Kugel, Jeff Libby, Scott Mackie, Swamy Sadashivaiah Renu Kananda, Chang-Hong Wu, and Manya Ghobadi. 2022. Using trio—juniper networks’ programmable chipset—for emerging in-network applications. In *Proceedings of the ACM SIGCOMM*.
- [128] Tong Yang, Jie Jiang, Peng Liu, Qun Huang, Junzhi Gong, Yang Zhou, Rui Miao, Xiaoming Li, and Steve Uhlig. 2018. Elastic sketch: Adaptive and fast network-wide measurements. In *Proceedings of the ACM SIGCOMM*.
- [129] Sophia Yoo and Xiaoqi Chen. 2021. Secure keyed hashing on programmable switches. In *Proceedings of the ACM SIGCOMM Workshop on Secure Programmable Network Infrastructure*. 16–22.
- [130] Minlan Yu, Lavanya Jose, and Rui Miao. 2013. Software defined traffic measurement with opensketch. In *Proceedings of the USENIX Conference on Networked Systems Design and Implementation*.
- [131] Yifan Yuan, Omar Alama, Jiawei Fei, Jacob Nelson, Dan R. K. Ports, Amedeo Sapio, Marco Canini, and Nam Sung Kim. 2022. Unlocking the power of inline floating-point operations on programmable switches. In *Proceedings of the USENIX Symposium on Networked Systems Design and Implementation*. 683–700.
- [132] Lior Zeno, Dan R. K. Ports, Jacob Nelson, Daehyeok Kim, Shir Landau Feibish, Idit Keidar, Arik Rinberg, Alon Rashelbach, Igor Lima de Paula, and Mark Silberstein. 2022. SwiSh: Distributed shared state abstractions for programmable switches. In *Proceedings of the USENIX Symposium on Networked Systems Design and Implementation*. USENIX Association, 171–191. Retrieved from <https://www.usenix.org/conference/nsdi22/presentation/zeno>
- [133] Menghao Zhang, Guanyu Li, Shicheng Wang, Chang Liu, Ang Chen, Hongxin Hu, Guofei Gu, Qi Li, Mingwei Xu, and Jianping Wu. 2020. Poseidon: Mitigating volumetric DDoS attacks with programmable switches. In *Proceedings of the IEEE Conference on Network and Distributed System Security*.
- [134] Xiaoquan Zhang, Lin Cui, Kaimin Wei, Fung Po Tso, Yangyang Ji, and Weijia Jia. 2021. A survey on stateful data plane in software defined networks. *Comput. Netw.* 184 (2021), 107597. DOI: <https://doi.org/10.1016/j.comnet.2020.107597>
- [135] Yinda Zhang, Peiqing Chen, and Zaoxing Liu. 2024. OctoSketch: Enabling real-time, continuous network monitoring over multiple cores. In *Proceedings of the USENIX Symposium on Networked Systems Design and Implementation (NSDI’24)*.
- [136] Yinda Zhang, Zaoxing Liu, Ruixin Wang, Tong Yang, Jizhou Li, Ruijie Miao, Peng Liu, Ruwen Zhang, and Junchen Jiang. 2021. CocoSketch: High-performance sketch-based measurement over arbitrary partial key query. In *Proceedings of the ACM SIGCOMM*.
- [137] Fuheng Zhao, Punnaal Ismail Khan, Divyakant Agrawal, Amr El Abbadi, Arpit Gupta, and Zaoxing Liu. 2023. Panakos: Chasing the tails for multidimensional data streams. *Proc. VLDB Endow.* 16, 6 (2023), 1291–1304.
- [138] Yikai Zhao, Kaicheng Yang, Zirui Liu, Tong Yang, Li Chen, Shiyi Liu, Naiqian Zheng, Ruixin Wang, Hanbo Wu, Yi Wang, and Nicholas Zhang. 2021. LightGuardian: A full-visibility, lightweight, in-band telemetry system using sketchlets. In *Proceedings of the USENIX Symposium on Networked Systems Design and Implementation*. USENIX Association, 991–1010. Retrieved from <https://www.usenix.org/conference/nsdi21/presentation/zhao>

- [139] Zhipeng Zhao, Hugo Sadok, Nirav Atre, James C. Hoe, Vyas Sekar, and Justine Sherry. 2020. Achieving 100Gbps intrusion prevention on a single server. In *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, 1083–1100.
- [140] Hao Zheng, Chen Tian, Tong Yang, Huiping Lin, Chang Liu, Zhaochen Zhang, Wanchun Dou, and Guihai Chen. 2022. FlyMon: Enabling on-the-fly task reconfiguration for network measurement. In *Proceedings of the ACM SIGCOMM (SIGCOMM'22)*. 486–502.
- [141] Yufei Zheng, Xiaoqi Chen, Mark Braverman, and Jennifer Rexford. 2022. Unbiased delay measurement in the data plane. In *Proceedings of the Symposium on Algorithmic Principles of Computer Systems (APOCS'22)*. SIAM, 15–30.
- [142] Yufei Zheng, Huacheng Yu, and Jennifer Rexford. 2022. Detecting TCP packet reordering in the data plane. Retrieved from <https://arXiv:2301.00058>
- [143] Yu Zhou, Dai Zhang, Kai Gao, Chen Sun, Jiamin Cao, Yangyang Wang, Mingwei Xu, and Jianping Wu. 2020. Newton: Intent-driven network traffic monitoring. In *Proceedings of the ACM SIGCOMM CoNEXT Conference*.

Received 22 January 2024; revised 11 November 2024; accepted 22 January 2025